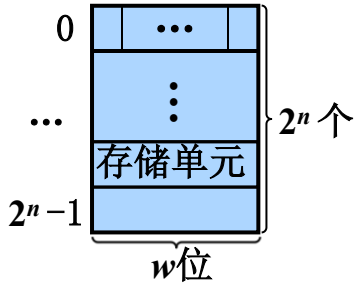
第三章　存储系统

SEU.CSE.RGL

※存储器回顾



基本结构—由定长单元构成的一维线性空间，
按地址访问

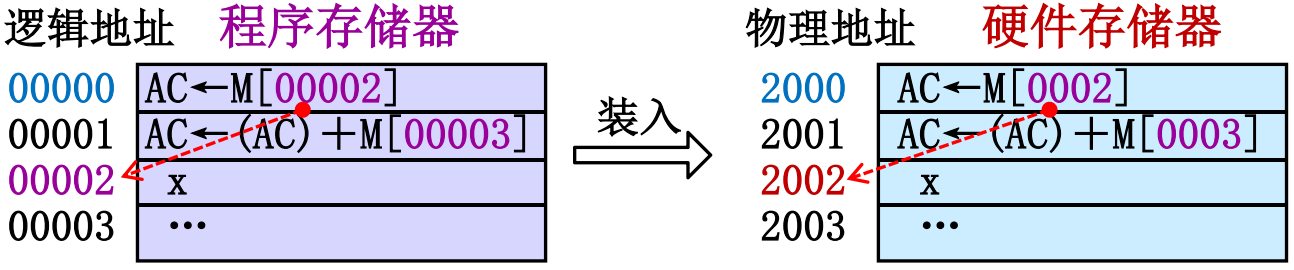
访问过程—接收地址及命令，选择单元、启动操作、输出数据
接收地址及命令、数据，选择单元、启动操作

应用需求—

软件：程序放在程序MEM中(从0开始、程序MEM空间>>硬件MEM空间)，
程序按逻辑地址执行(访问MEM/形成MEM地址)

硬件：程序装入MEM后才执行，装入位置可变(多个程序共用MEM)，
容量大-速度快-价格低

程序MEM：
指程序空间的MEM抽象，并不真实存在！



※主要内容

目标—组织高性/价的存储系统，可按程序地址访问

(1)层次结构存储系统　　(存储系统的结构)

MEM技术指标，MEM的层次结构(程序访问局部性、组成要求)

(2)半导体存储器　　(硬件基础)

SRAM的组成、读写时序，DRAM的组成、读写时序、刷新

(3)主存　　(设计与连接)

组成，逻辑设计，与CPU的连接，提高访存速度的技术(3类)

(4)高速缓存(Cache)　　(缓冲器的组成)

组成(空间管理、工作过程)，地址映射、替换算法、写策略的实现

(5)虚拟存储器(△)　　(按程序地址访存的组织)

组成(结构、工作过程)，VM的存储管理方式，VM的实现

※总体要求：可设计主存、连接主存，掌握缓冲器的组成原理

§3.1　存储系统概述

※主要内容：MEM的技术指标，存储系统的组成、工作过程

一、存储器分类

*按存储介质分类：　半导体器件、磁性材料、光介质

*按存取方式分类：

	顺序存取(SAM)	随机存取(RAM)	直接存取(DAM)	只读(ROM)
操作类型	存、取			取
存取时间	可变(取决于地址)	固定(与地址无关)	可变(与地址相关)	同RAM/DAM
编址单位	记录块	存储字	记录块(多个存储字)	同RAM/DAM

*按应用功能分类：

	主存(MM)	辅存(SM)	高速缓存(Cache)	控存(CS)
应用功能	CPU直接访问的MEM	主存的后援MEM	CPU-主存间的缓冲MEM	CPU内部存放微程序的MEM
存储介质	MOS型半导体	磁/光介质	MOS型半导体	MOS型半导体
存取方式	DRAM和ROM	SAM/DAM	SRAM	ROM

*其他分类：　易失型、非易失型等

二、存储器的主要技术指标

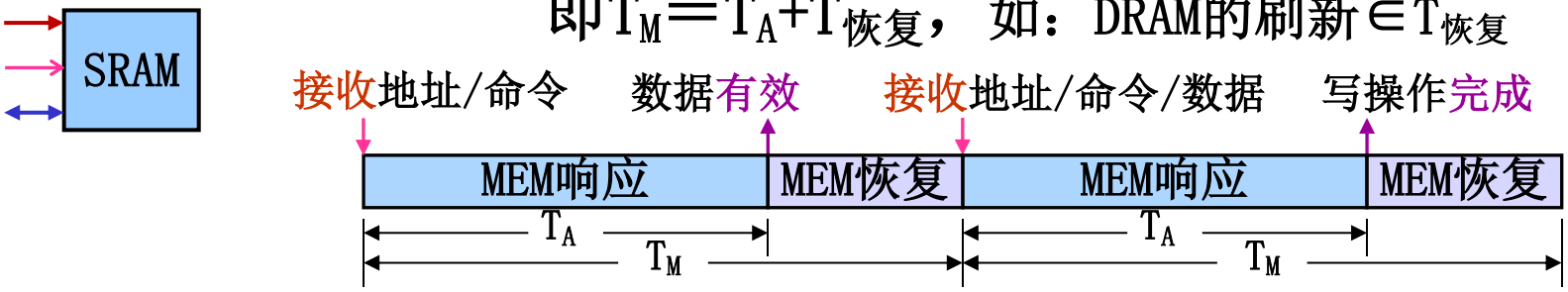
*存储容量(S)： 可存储的二进制位数，单位常为字节(B)

*存取速度： 常用访问时间、存取周期表示，单位为μs等

访问时间(T_A)— 指MEM收到命令→完成操作所需时间

存取周期(T_M)— 指MEM两次操作最短间隔时间

即$T_M = T_A + T_{恢复}$， 如：DRAM的刷新$\in T_{恢复}$



*传输速度： 常用MEM带宽表示，单位常为Mbps

MEM带宽(B_M)— 指MEM的最大数据传输率(传输2个数据的最小间隔)

如：基本RAM的$B_M = W/T_M$，多体交叉RAM的$B_M = n \cdot W/T_M$

其中，W为数据引脚位数，n为MEM体数

类比：同学需回答2个问题，T_A=听清问题→张嘴回答，$T_{恢复}$=题间喘气，B_M=语速

三、层次结构存储系统

1、层次结构的引入　（即为什么要采用层次结构）

*MEM的用户需求：大容量、高速度、低价格　（无理，但要满足!）

*程序访问的局部性原理（规律）：　（救命稻草！）

程序运行时，指令和数据访问所呈现出的相对簇聚现象

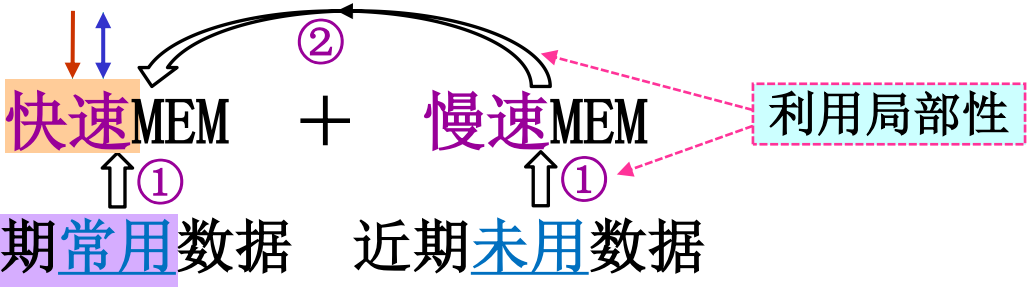
示例：for (i=0; i<n; i++) S=S+A[i];

时间局部性— 最近访问过的信息，将再次被访问

空间局部性— 最近访问信息的相邻信息，将很快被访问

*用户需求的解决方案：

MEM组成— 层次结构的 快速MEM ＋ 慢速MEM



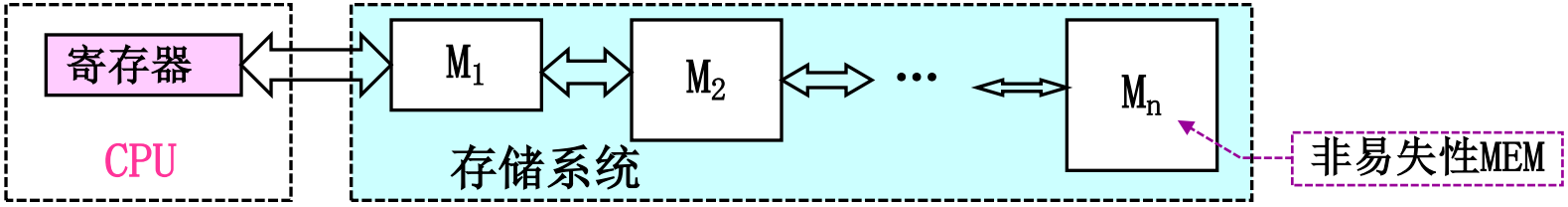
速度保证—　近期常用数据　近期未用数据

容量保证—　容量小　容量大 ＝ 容量大

价格保证—　占比小 ＋ 占比大 ＝ 价格低

2、存储系统的层次结构

*结构与组成：多种MEM级联，按速度分层、协调工作



参数要求— $S_{Mn}>>\cdots>>S_{M2}>>S_{M1}$，$T_{Mn}>>\cdots>>T_{M2}>>T_{M1}$

目标：$(c_1S_{M1}+c_2S_{M2}+\cdots+c_nS_{Mn})/(S_{M1}+S_{M2}+\cdots+S_{Mn}) \approx c_n$

原因：① S_{Mi}级差大利于c_nS_{Mn}作为主因子

② T_{Mi}级差大利于减小c_iS_{Mi} ←T_{Mi}与c_i呈反比

*内部工作协调：

信息存放— 放在M_n中，M_i中信息为M_{i+1}中信息的副本

M_i改写后需写回M_{i+1}←┤

有效容量$S=S_{Mn}$ ←┘

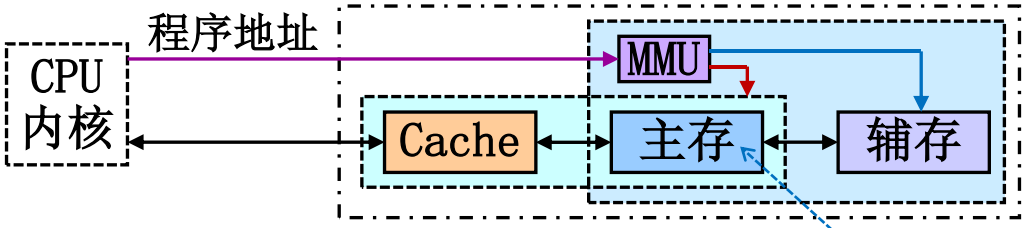
信息传递— 外部只访问M_1，内部各M_i间透明地传递信息

└← T_A不固定←—外部不可见←┘

*常见的层次结构： 以主存为中心

背景—程序存放在辅存中，执行前装入主存，执行时按程序地址访问

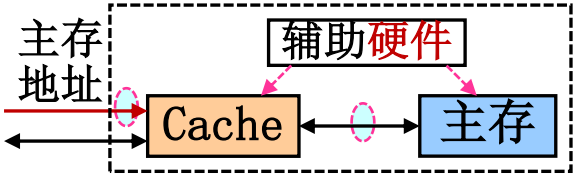
CPU只直接访问主存←┘←┘ （先变换成主存地址）



MMU—Memory Management Unit

程序空间>>主存空间

Cache-主存层次— （解决主存速度问题）

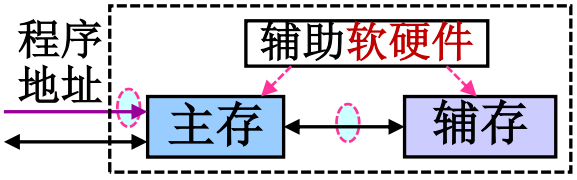


辅助①：地址变换（主存地址→Cache地址）

辅助②：层次管理（主存-Cache间透明传递）

实现： 地址变换、层次管理全部由硬件实现

主存-辅存层次— （解决主存容量问题）



实现：地址变换由硬件实现，

└←需减小变换开销

层次管理由软件实现

程序空间太大，硬件管理成本太高→┤

常实现为虚拟存储器（OS负责）→┘ （早期程序员负责）

§3.2 半导体存储器技术

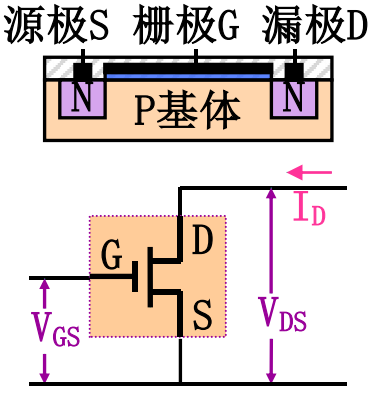
※主要内容：SRAM及DRAM芯片的组成、读写时序(及DRAM刷新)

*半导体存储器分类：

- 半导体RAM(易失型)
  - 双极型RAM(TTL、ECL)
  - MOS型RAM
    - 静态RAM(SRAM)
    - 动态RAM(DRAM)
- 半导体ROM(非易失型)
  - MROM、PROM、EPROM、EEPROM(E²PROM)，FLASH

思考：SRAM、DRAM存储信息的元件？

*MOS管简介：绝缘栅场效应管(Metal Oxide Semiconductor, MOS)



MOS管特性—

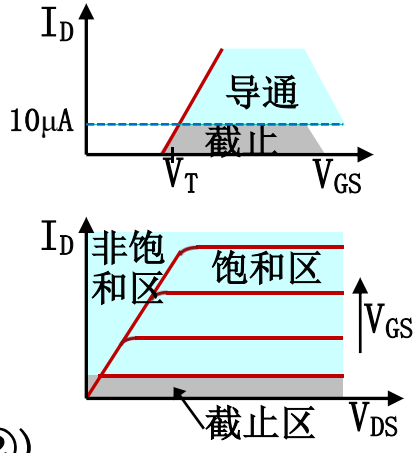
①$V_{GS}<V_T$时，$I_D=0$(MOS管截止)

$V_{GS}\geqslant V_T$时，$I_D>0$(MOS管导通)

②V_{GS}越大时，I_D越大(与V_{DS}无关)

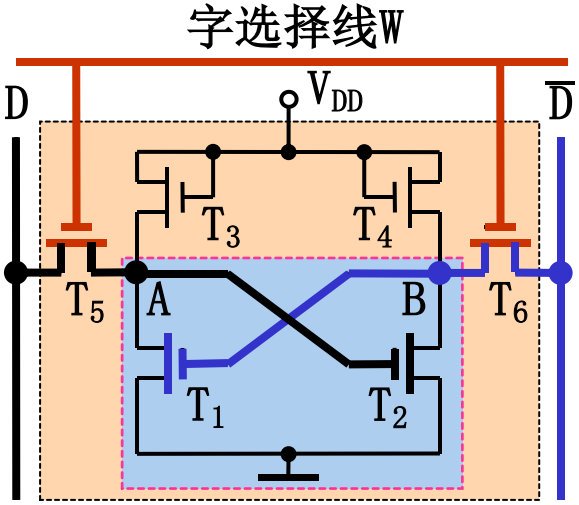
③V_{GS}固定、增大V_{DS}时，I_D不变

MOS管应用— 用作开关(①)、电阻(②)



一、静态RAM(Static RAM, SRAM)

1、SRAM存储元的组成原理 (了解操作方法)



MOS型静态存储元组成

注：T_3和T_4—用作上拉电阻，T_5和T_6—用作开关

*存储原理：若T_1导通→$V_A=V_{地}$→T_2截止

→$V_B=V_{高}$→T_1更导通(稳态)

*写入方法：①W线上加正脉冲(宽度≥写延迟)

→T_5和T_6导通(写入期间)；

②写0时，使$D=V_{地}$、$\overline{D}=V_{中}$→T_2截止、T_1导通；写1时反之

*保持方法：使$V_W=V_{地}$→T_5和T_6截止→T_1及T_2状态保持不变($V_B=V_{高}$)

*读出方法：①W线上加正脉冲→T_5和T_6导通(读出期间)；

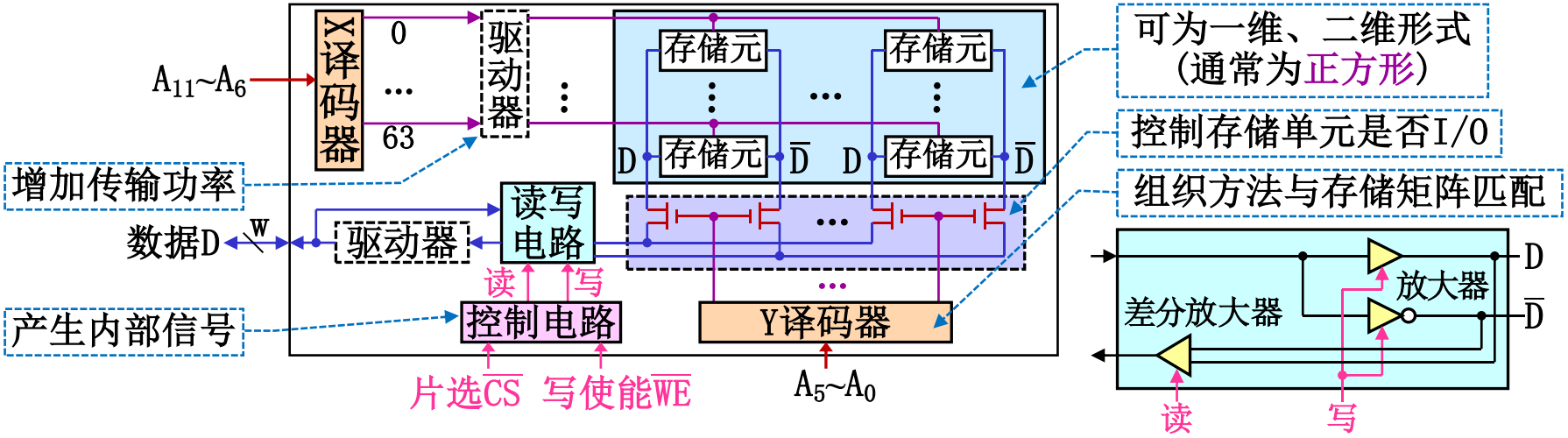
②使$D=\overline{D}=V_{中}'$→D或$\overline{D}$上电压产生变化(0时D电压下降)

→用差分放大器可检测出所存信息→T_1及T_2状态不变(非破坏性读)

2、SRAM芯片的组成

(1)SRAM基本组成

包括存储矩阵、地址译码器、I/O门、读写电路、控制电路等



思考①：上图w＝？

思考②：w＞1时的存储矩阵、译码器如何组织？

通常三维矩阵(w个位平面)、二维译码(同时控制w个位平面)

思考③：为什么要设置$\overline{CS}$引脚？控制电路的逻辑是？

SRAM状态有读/写/无操作3种，需2根引脚； 读＝$(\sim\overline{CS})\cdot\overline{WE}$，读＝$(\sim\overline{CS})\cdot(\sim\overline{WE})$

(2)SRAM引脚组织

若存储单元长度为w、存储单元个数为2^n，则容量$S=2^n\times w$

*参数标识：地址空间×数据位数，如1K×8b SRAM

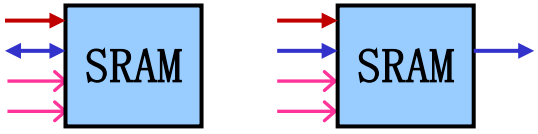
*引脚组织：

数据引脚— w位，有单向、双向2种方式(分别为$2w$根、w根)

地址引脚— $\log_2(S/w)=n$根 ←地址总是单向的

控制引脚— 片选(1根)+读/写(1根) ←状态有3种[读/写/闲]

←有效电平可为高/低



思考①：某SRAM的容量为4Kbit、数据引脚(双向)有8根，地址引脚为多少根？若数据引脚改为32根，地址引脚为多少根？

思考②：某SRAM的数据引脚(单向)为8根、地址引脚为8根，该芯片的容量为多少个字节？

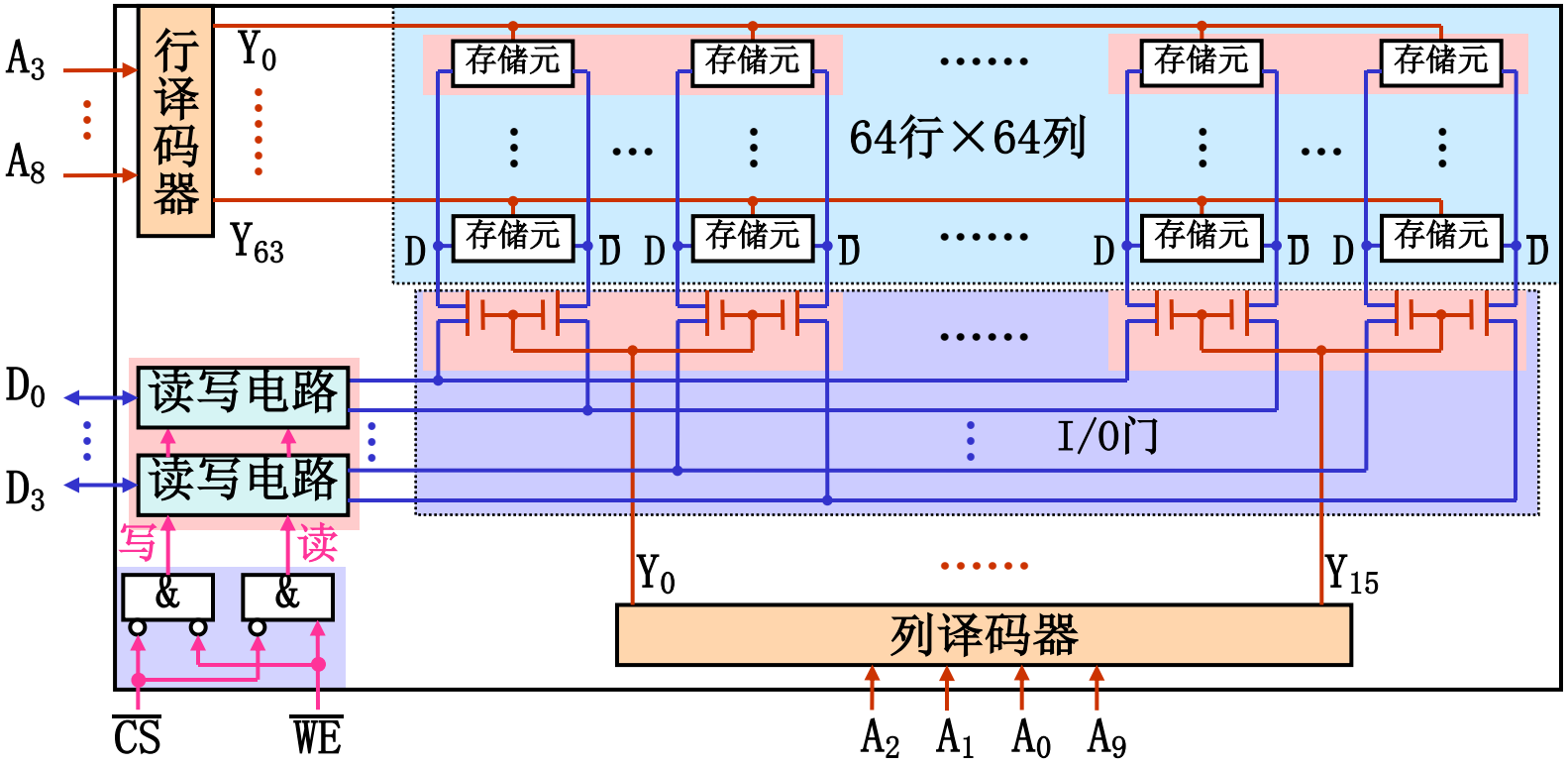
思考③：1K×4位SRAM中，除电源信号线外，其余信号线至少多少根？

*组成示例： Intel 2114 SRAM芯片(很久以前)

参数—容量为1K×4b，双向数据引脚

引脚—数据(4根)，地址(10根)，控制(片选+读/写)

组成—存储矩阵(64×64)，I/O门(4位)，读写电路(4位)，地址译码($\log_2 64+\log_2[64/4]$)，控制电路

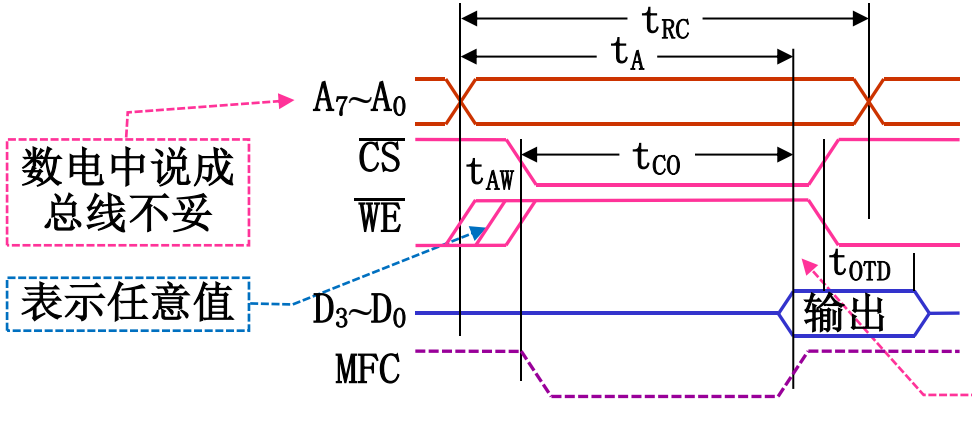


3、SRAM芯片的读写时序　(芯片对引脚信号的时序要求)

*操作的标志：开始—CS#从无效→有效　停止—CS#从有效→无效

*读周期的时序：　(厂家给出的参数是最小值)

约定：CS#表示$\overline{CS}$(便于书写)



t_{AW} —读滞后时间(=译码时延)

t_{CO} —开始操作→数据输出稳定

t_{OTD}—停止操作→数据引脚高阻

t_A —读时间($\geq t_{AW}+t_{CO}$)

t_{RC} —读周期($\geq t_{AW}+t_{CO}+t_{传送}$)

操作控制的信号时序要求—

①地址先发送，CS#在地址稳定后有效　←对指定单元操作

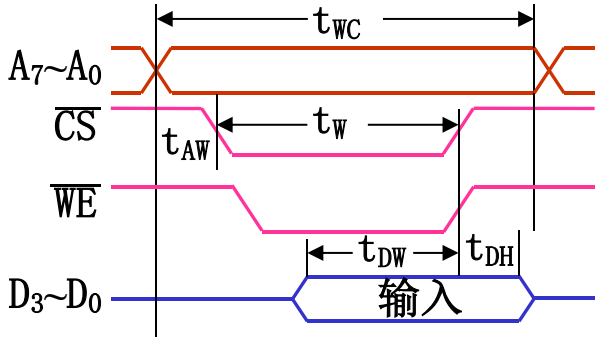
思考①：CS#有效后，地址还在抖动，t_{CO}时能输出结果吗？　能(单元未知)

②WE#在CS#有效期间无效(读)　←防止误写入

思考②：CS#有效前，对WE#的值有要求吗？　无要求(CS#无效时无动作)

操作完成的表示—约定时延，或用状态线表示(如MFC)

*写周期的时序：　（厂家给出的参数是最小值）



t_{AW} —写滞后时间(＝译码时延)
t_{DW} —数据写入到存储单元时间
t_{DH} —$\overline{WE}$无效后数据需保持时间
t_W —写入时间($\geq t_{DW}$)
t_{WC} —写周期($\geq t_{AW}+t_A$)

操作信号的时序要求—

①地址先发送，$\overline{CS}$#在地址稳定后有效　←对指定单元操作

②WE#在CS#有效期间有效时长$\geq t_{DW}$　←保证写完成

└←时刻无要求(CS#有效后才会写)

③数据在CS#与WE#均有效期间保持时长$\geq t_{DW}$　←时刻无要求

思考：读/写控制要求合并后，总的要求是？　R操作有此要求

地址先发送；$\overline{CS}$在地址稳定后有效，$\overline{WE}$早于/等于$\overline{CS}$无效(R)/有效(W)；

数据(W时)发送时刻无要求、保持时长$\geq t_{DW}$($\overline{CS}$及$\overline{WE}$均有效时)

*SRAM的性能：$T_A = t_{AW}+t_{CO}$或$t_{AW}+t_{DW}$，$T_M = t_{RC}$或t_{WC}

※疑惑：好抽象啊，为什么要学RAM？重点内容是什么？如何学习？

目的— 主存由SRAM/DRAM组成，设计主存时会用到

重点— 芯片的引脚组织(容量-引脚的关系)，

芯片的操作信号时序(读/写周期时序)

方法— 理解概念(看得懂描述)、面向应用(关注引脚组织/访问控制)

※SRAM的学习要求：

1、熟记SRAM的功能(信息存储、按地址访问[时延相同]) ←应用需求

(=$2^n \times w$)容量参数←┘ └→工作状态(=读/写/闲)

2、了解SRAM的内部组成(存储矩阵+译码器+I/O门+读写电路+控制电路)

[选行] [选列]

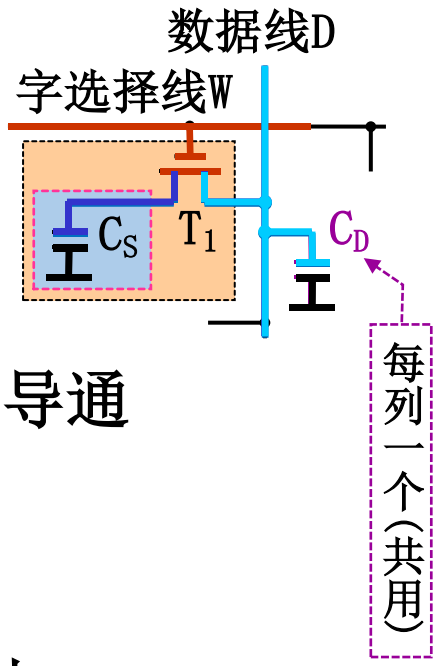
3、掌握SRAM的引脚组织方法(地址+数据+控制) ←容量-引脚关系

4、理解SRAM的操作信号时序(先发地址、读/写略不同) ←访问的实现

二、动态RAM(Dynamic RAM, DRAM)

DRAM的目标—降低功耗、节约成本

1、DRAM存储元的组成原理 --单管MOS型



*存储原理：无放电回路(T_1截止)时，电容中的电荷可以保持(会缓慢泄漏→需定时刷新)

*写入方法：①在W线上加正脉冲(宽度≥写延迟)→T_1导通

②在D线上加数据→对C_S充电/放电

*保持方法：使W＝$V_{地}$→T_1截止→C_S状态保持不变

*读出方法：①在D线上加正脉冲($=V_{中}$)→对C_D预充电

②在W线上加正脉冲→T_1导通→D线上电压产生变化

→用放大器读出数据→C_S状态发生改变(破坏性读)

→用所读数据重新写C_S ←称为再生

*刷新方法：同读操作(数据不输出到引脚) ←需求为读出后写入

2、DRAM芯片的组成

(1)DRAM芯片的引脚组织

*数据引脚：w位，有单向、双向2种方式

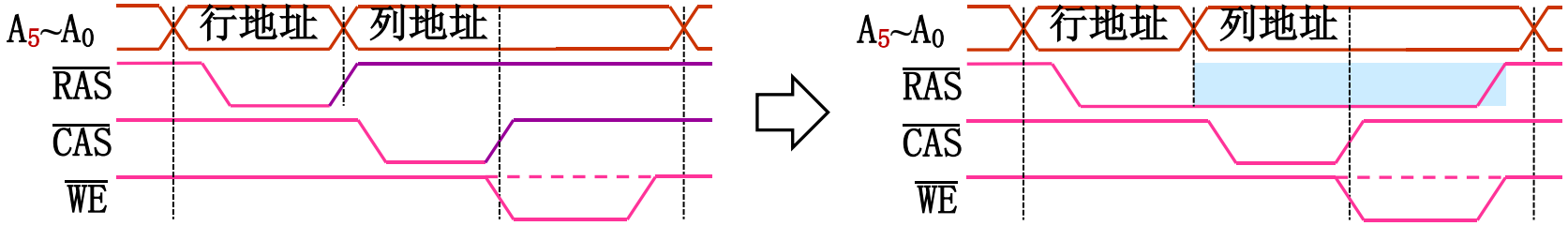
*地址引脚：$[\log_2(S/w)]/2$根，地址分2次传送(行地址＋列地址)

└←减少芯片面积←提高集成度

*控制引脚：行地址选通$\overline{RAS}$、列地址选通$\overline{CAS}$、读/写$\overline{WE}$

(Row Address Strobe) (Column Address Strobe)

片选功能的组织— 借用$\overline{RAS}$实现(操作期间全部有效)



思考：不设置RAS#、CAS#，可以表示A_5~A_0当前是行/列地址码？

WE#中E的英文单词是什么？

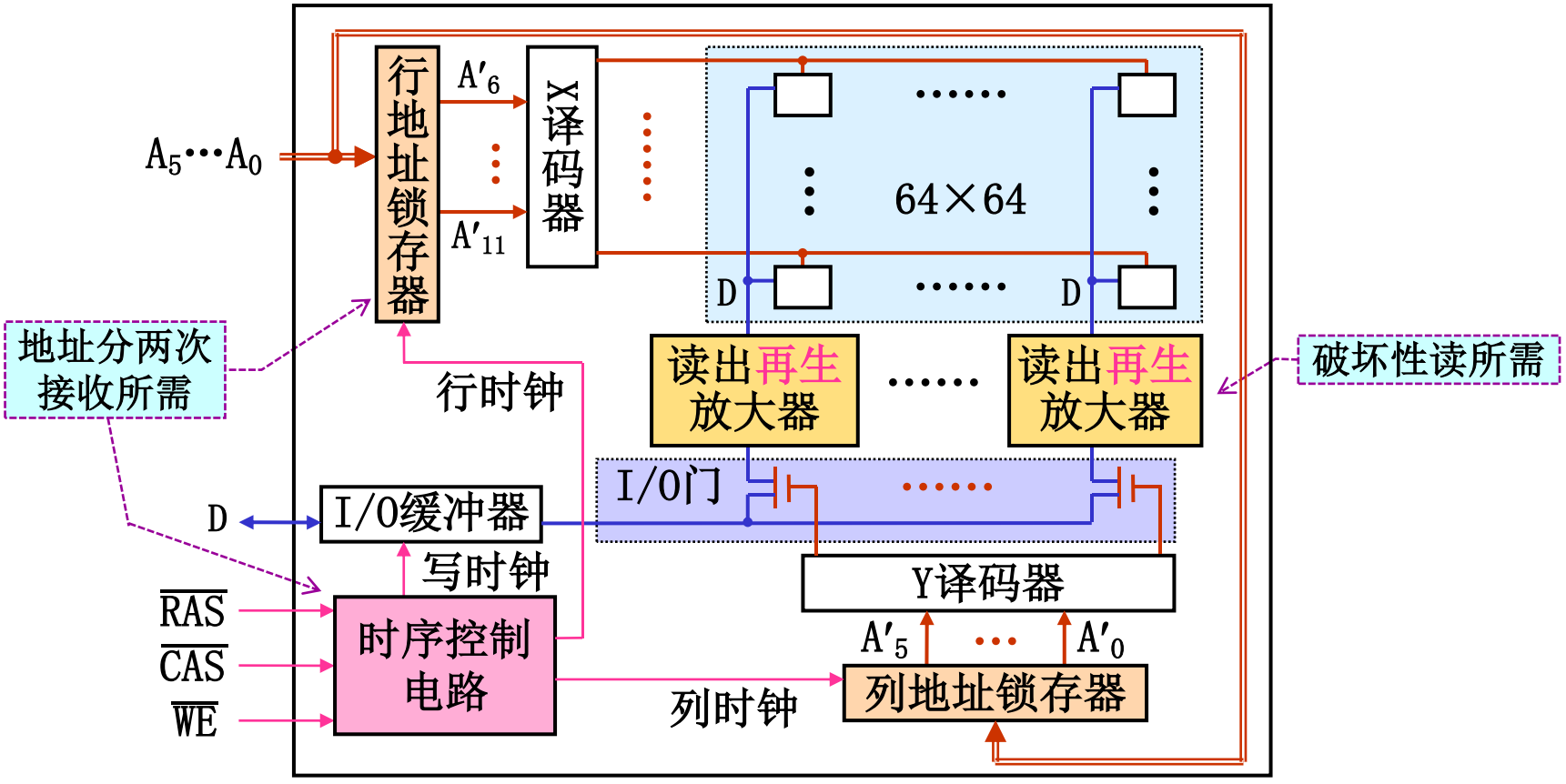
刷新功能的组织— 稍后讨论

(2)DRAM芯片的基本组成

包括存储矩阵、地址译码器、I/O门、读写电路，←类似于SRAM

以及地址锁存器、时序控制电路、再生电路　　←与SRAM不同

（边沿触发）　　（3个信号串联）



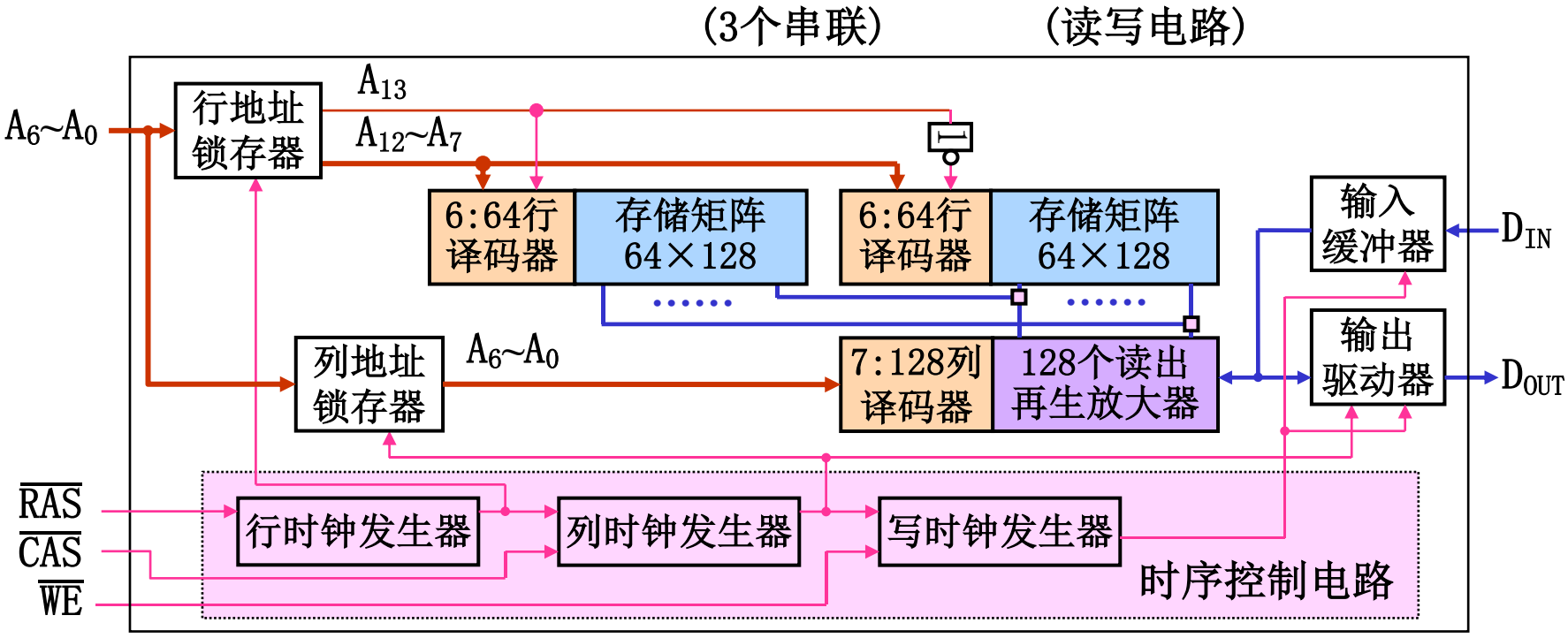
*组成示例：Intel 2116 DRAM芯片

参数—容量为16K×1b，单向数据引脚

引脚—数据(2根)，地址(7根)，控制(行选+列选+读/写)

组成—存储矩阵(2×64×128)，地址译码器；

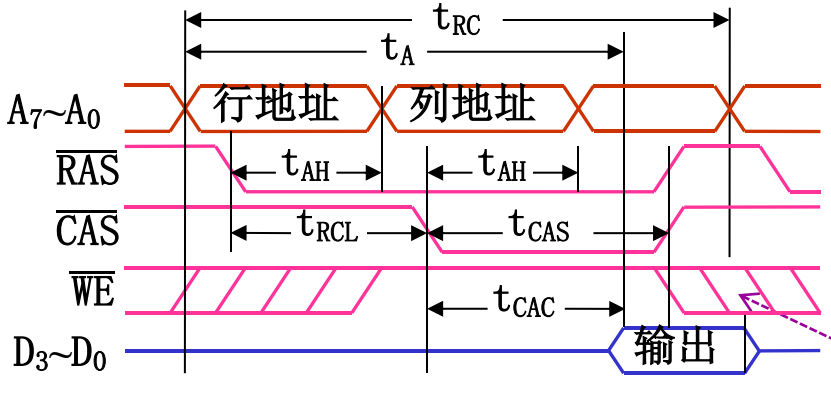
地址锁存器、时序控制电路，再生放大器



3、DRAM芯片的操作时序　　(芯片对引脚信号的时序要求)

***操作的标志：** $\overline{RAS}$从无效→有效、从有效→无效

***读周期的时序：** (厂家给出的参数为最小值)



t_{AH} —地址锁存延迟

t_{RCL}--$\overline{RAS}$与$\overline{CAS}$信号延迟

t_{CAC}—$\overline{CAS}$有效→数据输出稳定

t_A —读时间($\geq t_{RCL}+t_{CAC}$)

t_{RC}—读周期($\geq t_A+\max\{t_{再生}, t_{传输}\}$)

操作信号时序— ①行地址先发送，$\overline{RAS}$在行地址稳定后有效

②列地址后发送，$\overline{CAS}$在列地址稳定后有效

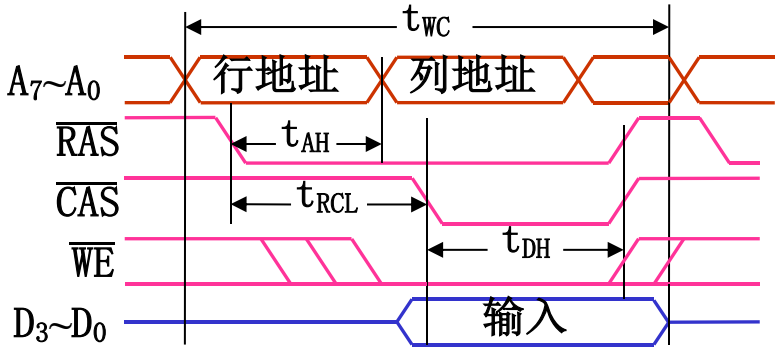
③$\overline{WE}$在$\overline{CAS}$有效期间无效(R)

例—某DRAM芯片的t_{AH}=0.4μs、t_{CAC}=3.2μs，若访存时操作信号都与1MHz的时钟脉冲同步(在时钟上升沿时发出)，则访问时间t_A至少为多少？

答：t_C=1μs，$t_A \geq \{1+\lceil 0.4/t_C \rceil+1+\lceil 3.2/t_C \rceil\} \times t_C$=7μs(>3.6μs)

(发行地址)(发RAS)(发列地址)(发CAS&等待完成)

*写周期的时序：　　(厂家给出的参数为最小值)



t_{AH} —同DRAM读周期

t_{RCL}—同DRAM读周期

t_{DH} —数据写入时间(同SRAM的t_{DW})

t_A —写时间(=t_{RCL}+t_{DH})

t_{WC}—写周期(两次间隔)

操作信号时序—

①先发送行地址，$\overline{RAS}$在行地址稳定后有效

②再发送列地址，$\overline{CAS}$在列地址稳定后有效

③$\overline{WE}$在$\overline{CAS}$有效期间有效(W)　←延迟最小化(可滞后、时长≥t_{DH})

④数据发送可早于$\overline{CAS}$　←延迟最小化(可滞后、时长≥t_{DH})

4、DRAM芯片的刷新

(1)芯片的刷新操作

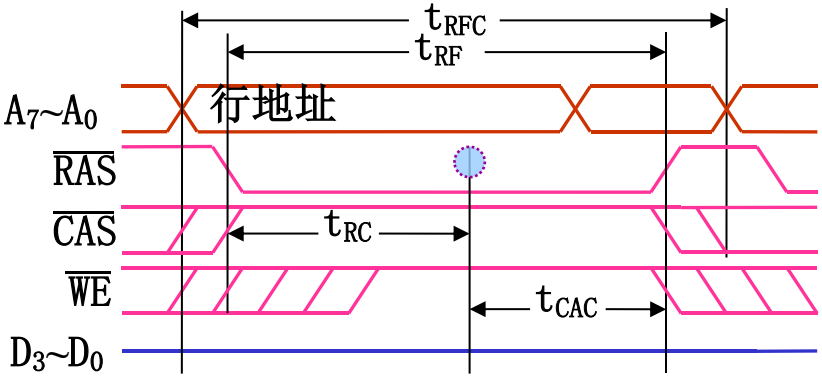
刷新需求—读出后写入(同读操作)、数据不输出到引脚

(列译码器输出全部无效)

*刷新操作的功能：刷新同一行的所有存储元(行刷新)

*刷新周期的时序：类似于读周期，$\overline{CAS}$一直无效

└→$\overline{WE}$值任意(时钟发生器串联)



t_{RC}--$\overline{RAS}$与$\overline{CAS}$信号最大延迟

t_{CAC}—$\overline{CAS}$有效→数据输出稳定

t_{RF} —刷新时间(t_A最大值)

t_{RFC}—刷新周期(两次间隔)

*刷新操作的组织：用定时器检测t_{RC}，$\overline{CAS}$无效时为刷新

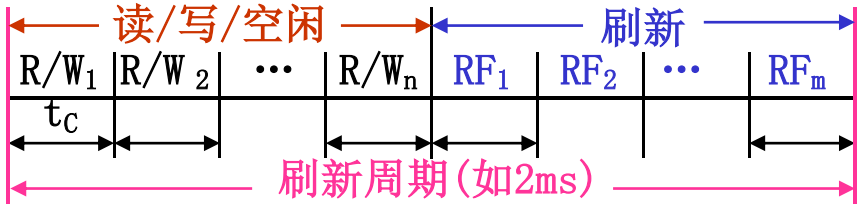
└→无需设置刷新引脚

(2) 芯片的刷新方式

*刷新周期：指同一存储元刷新的最大间隔时间 ←同一行的刷新间隔

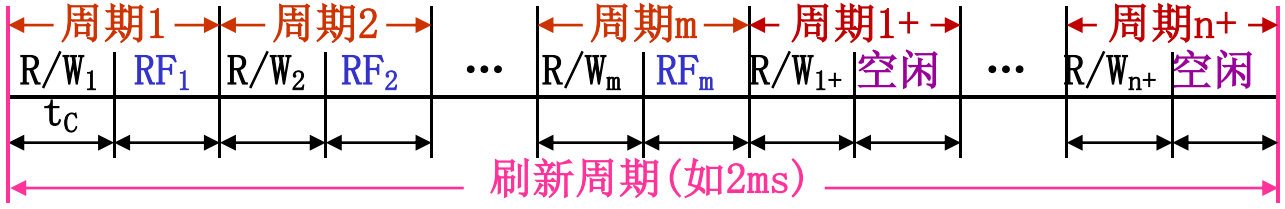
刷新方式—指不同行刷新的间隔特征

*集中式刷新：各行刷新连续进行，存在死区、T_M不变

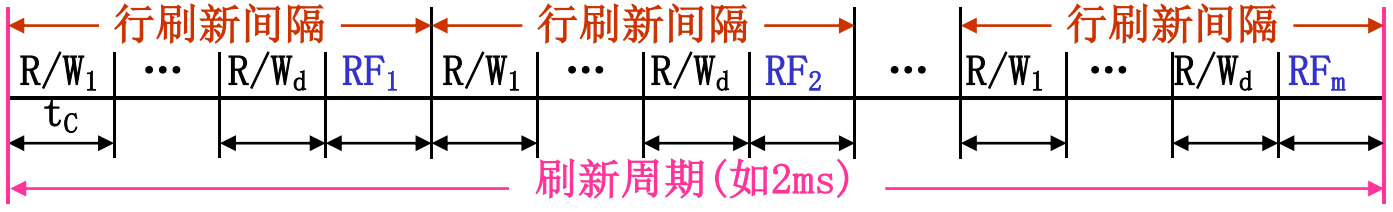


tC—存取周期
m—行刷新次数
n+m—可访存总次数

*分散式刷新：各行刷新分散在每个T_M中，无死区、T_M翻倍



★*异步式刷新：各行刷新均匀分布在刷新周期中，无死区、T_M不变



(3) 芯片刷新的实现

通常用外部电路实现，如DRAM控制器(DRAMC)

└←产生刷新操作命令　　└←又称主存控制器

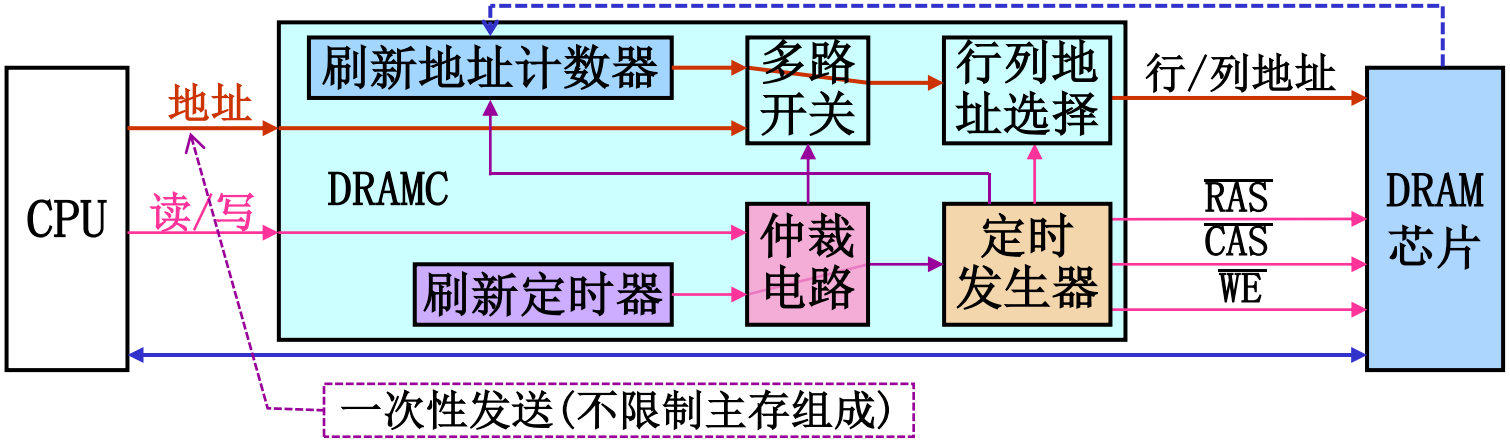
*DRAMC的主要功能：管理对主存的操作、实现对DRAM的刷新

*DRAMC功能的组织：

行刷新的地址—计数器产生，初值(行数)在系统启动时设置

行刷新的时机—定时器实现，刷新方式已设定(刷新定时器中)

操作的管理—仲裁电路实现(刷新的优先级较高)



5、DRAM与SRAM的比较

***DRAM的特点：** --相对于SRAM

组成— 存储元的元件少，地址引脚减半

优点— 集成度高、功耗低(1/4)、成本低(1/100)

缺点— 速度慢(1/10) ←使用电容所致

***RAM的应用：**

SRAM— 构成高速度MEM(容量不大)， 如Cache

DRAM— 构成大容量MEM(速度一般)， 如主存

※提醒： 学以致用→┬←目标是主存的设计、操作控制
↓
掌握芯片的引脚组织、操作信号时序！

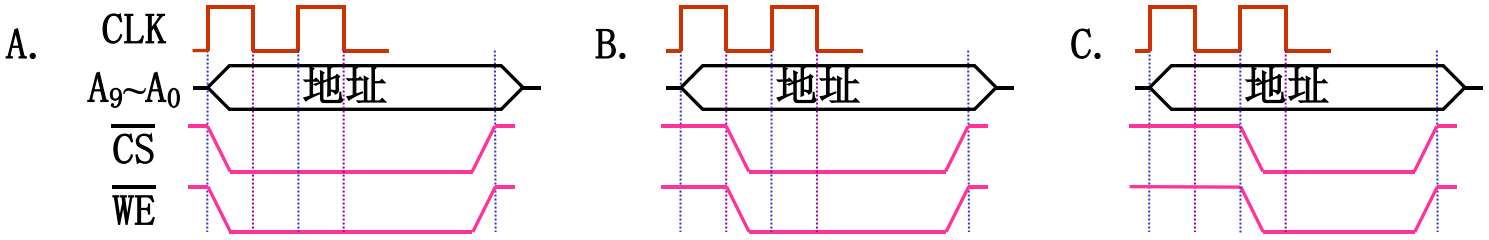
作业3-1：P137—3、4、6

※本节课堂练习：

1、1M×4b SRAM芯片的引脚(电源除外)至少为

A. 24根　　B. 25根　　C. 26根　　D. 27根

2、访问RAM时，若地址、命令信号的发送都与时钟信号CLK同步，则下列选项中，最优的实现方法是



3、1M×4b DRAM芯片的引脚(电源除外)至少为

A. 14根　　B. 15根　　C. 16根　　D. 17根

4、若1M×1b DRAM芯片的存取周期为1μs、刷新周期为8ms，采用异步刷新方式，则2次行刷新之间可操作的次数最多为

A. 6976/1024　　B. 6976/1000

C. 8000/1024　　D. 8000/1000

§3.3 主存储器

※主要内容：主存逻辑设计，主存与CPU连接，提高访存速度的技术

一、主存的组成

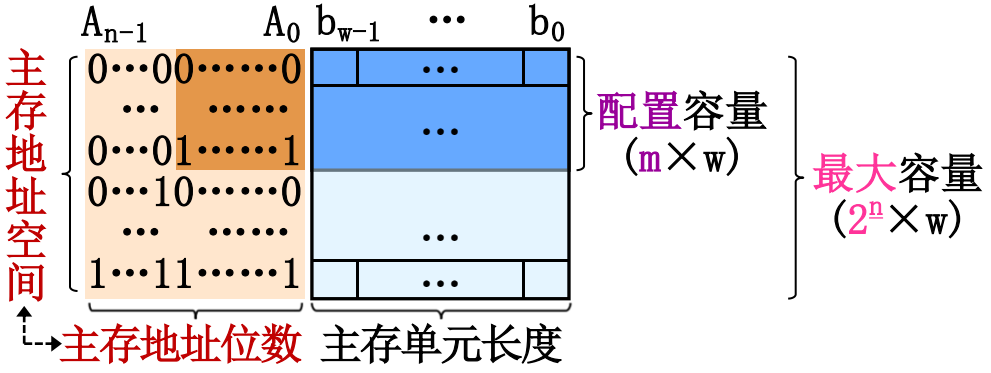
*主存的组成：ROM＋RAM，容量＝主存单元个数×主存单元长度

└←启动时需使用　└←大小可选择　└←ISA已确定

*主存的参数：

主存地址空间—主存单元个数最大值，系统设计时确定

主存单元个数—≤主存地址空间，用户配置时确定



二、主存的逻辑设计

*ROM/RAM芯片的现状：

容量参数—＝存储单元长度×存储单元个数

＝存储字长×存储字数

芯片规格—型号有限　　←销量-成本的平衡

*主存逻辑设计的方法：

用ROM、RAM(SRAM或DRAM)芯片进行容量扩展，实现指定的参数

*MEM的容量扩展方法：位扩展法、字扩展法、字位扩展法

步骤— ①确定需要多少芯片

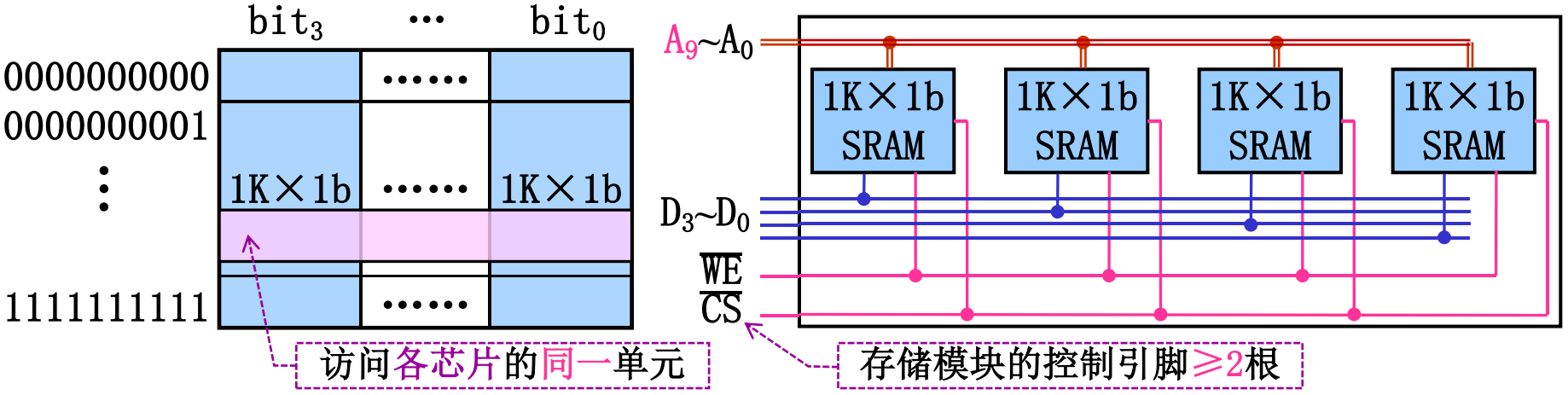
②明确芯片所占主存空间(所处位置)

③根据访问特征连接芯片

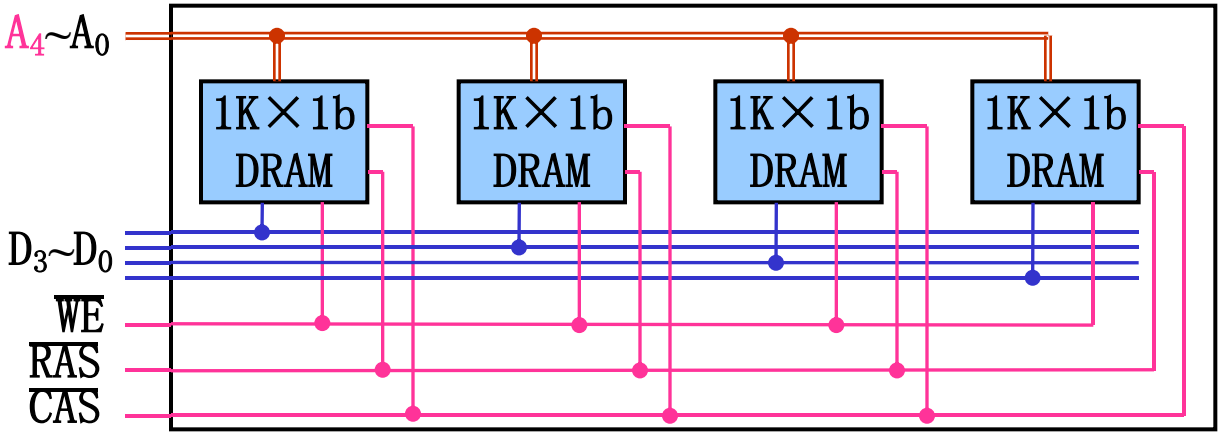
1、位扩展法　　—扩展存储字长(存储字数不变)

*芯片连接：数据引脚连接不同(组成存储字)，其余相同(并联)

例1—用1K×1b SRAM芯片构成1K×4b的存储模块



例2—用1K×1b DRAM芯片构成1K×4b存储模块



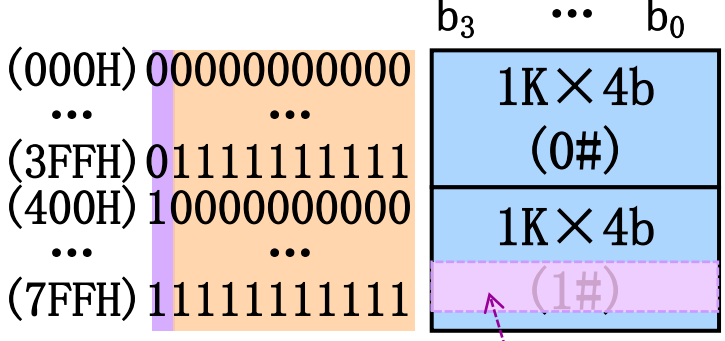
2、字扩展法　　—扩展存储字数、存储字长不变

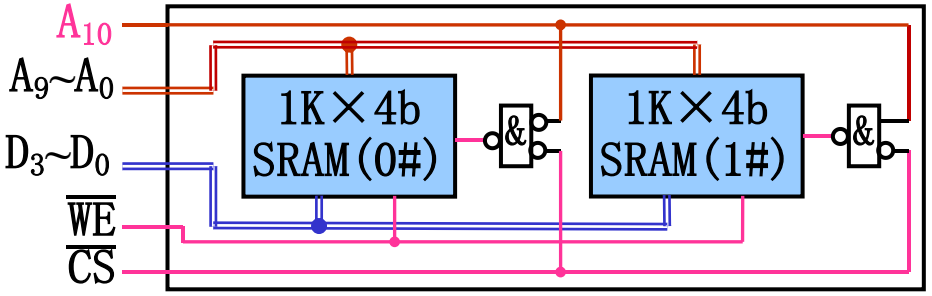
*芯片连接：片选引脚连接不同(选择存储字)，其余相同(并联)

例3—用1K×4b SRAM芯片构成2K×4b的存储模块

①芯片数量— (2K÷1K)×(4b÷4b)＝2片

②芯片所占空间— 地址范围不同，模块地址$\log_2(2K)$＝11位





访问某个芯片单元(地址＝模块地址低位)

③芯片连接— 直连数据、地址(模块地址低位)、读/写引脚

0#、1#芯片片选的逻辑为$\overline{\overline{CS}\cdot\overline{A_{10}}}$、$\overline{CS\cdot A_{10}}$

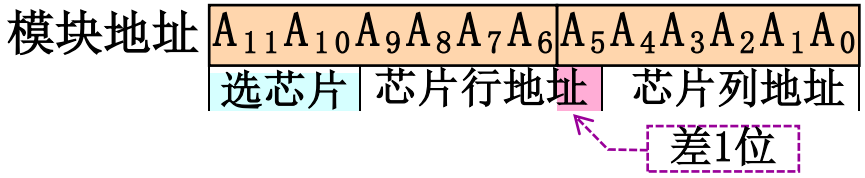
思考：SRAM位扩展、字扩展的芯片连接方法为何不同？

练习1—用1M×4b SRAM芯片构成4M×4b的存储模块

例4—用1K×4b DRAM芯片构成4K×4b的存储模块

解：所需芯片数量—(4K÷1K)×(4b÷4b)＝4片

扩展方法分析—无法用行地址选择芯片(行地址剩余长度不够)

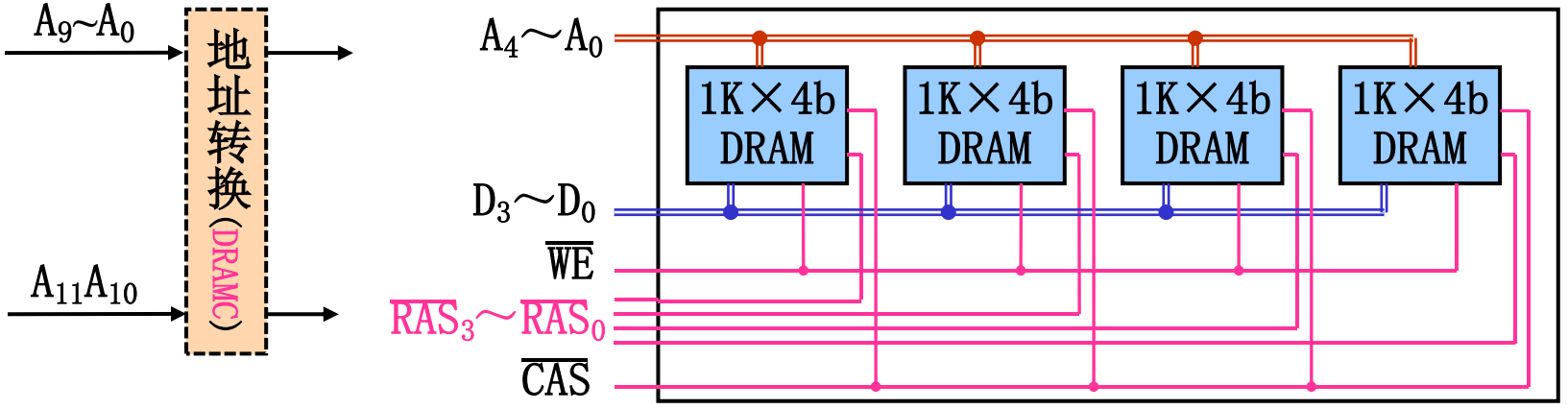


×扩展方法①—锁存行列地址　(适于芯片内部的体扩展)

增设2个锁存器，锁存后再选择芯片　←与芯片重复/成本高

☆扩展方法②—开放片选信号　(适于芯片外部的字扩展)

设置多个$\overline{RAS}$信号线，外部用地址高位选择芯片



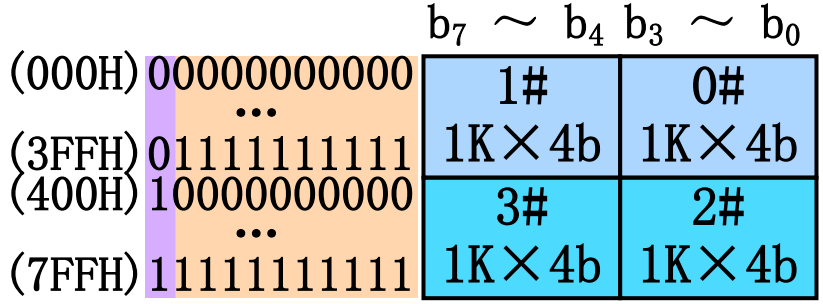
3、字位扩展法　　—扩展存储字数、存储字长

*芯片连接：先位扩展、再字扩展

例5—用1K×4b SRAM芯片构成2K×8b的存储模块

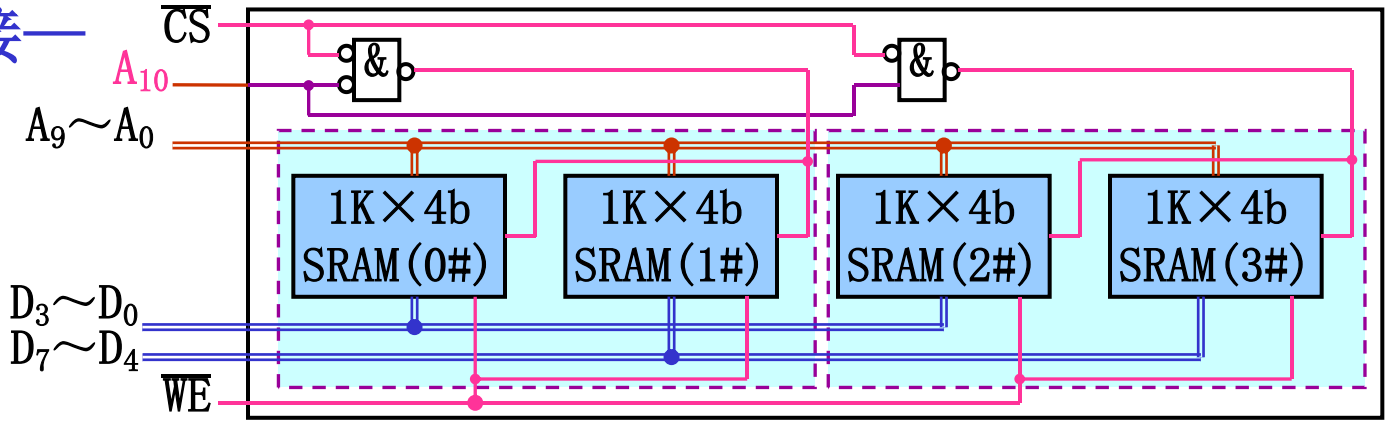
解：①芯片数量— (2K÷1K)×(8b÷4b)＝4片

②芯片所占空间— 地址/字内范围均不同，模块地址为11位



芯片	片选信号的逻辑
0#、1#	$\overline{CS}\cdot A_{10}$
2#、3#	$\overline{CS}\cdot A_{10}$

③芯片连接—



练习2—用1K×4b SRAM芯片构成4K×8b的存储模块

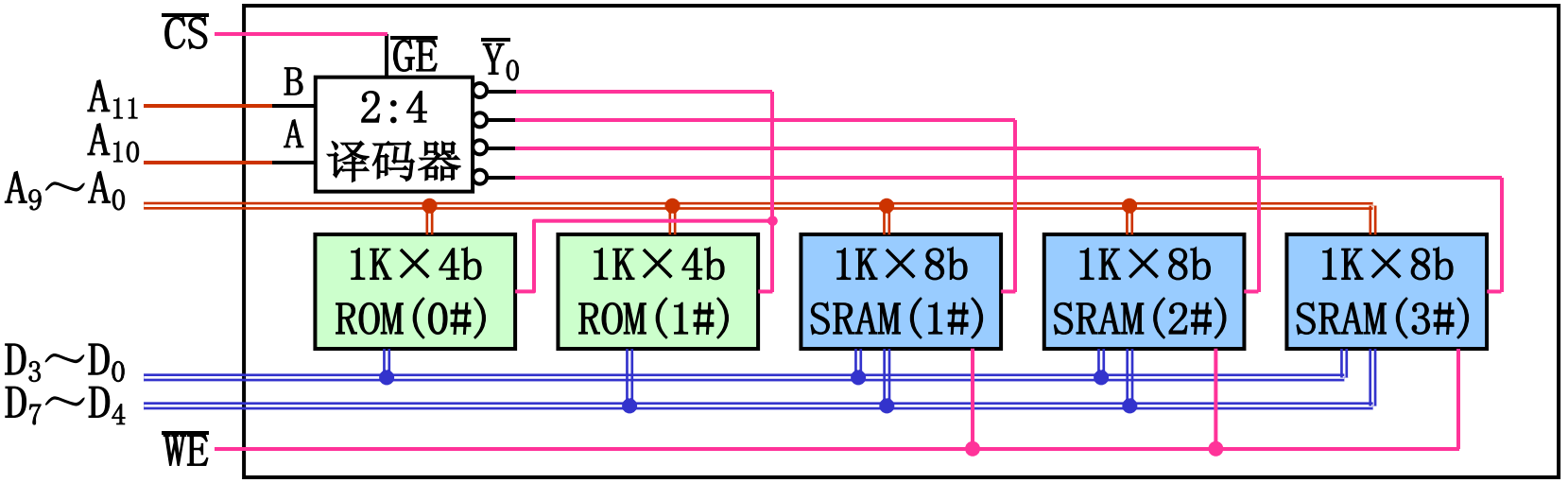
例6—用1K×4b ROM、1K×8b SRAM芯片构成4K×8b存储模块，其中前1KB空间为只读空间

解：①所需芯片数量— ROM需2片，SRAM需3片

②芯片所占空间— 模块地址为12位

③芯片连接— 先位扩展后字扩展

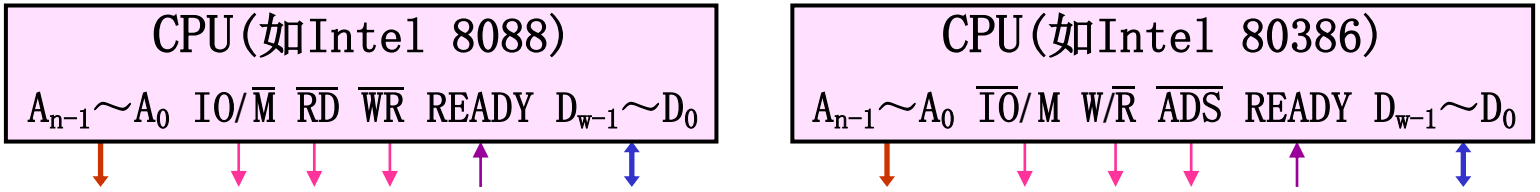
00XXXXXXXXX	ROM 1#	ROM 0#
01XXXXXXXXX	SRAM 1#	
10XXXXXXXXX	SRAM 2#	
11XXXXXXXXX	SRAM 3#	



三、主存与CPU的连接

1、CPU的存储器接口

外部访问需求—按地址访问存储器和I/O设备



*地址线：位数n＝可寻址空间大小，常＝主存地址位数(最大容量时)

*数据线：位数w应＝主存单元长度，可＝$k\times w$(提高性能，n相应缩小)

保持大小不变

*控制线：位数可表示所有操作类型、响应状态

操作类型— MEM操作(如读/写)、IO操作(如I/O)，无操作 (≥3根)

响应状态— 就绪/等待 (≥1根)

8088信号功能约定：IO/$\overline{M}$—访问对象类型(低为MEM)，$\overline{RD}$—MEM读或外设In，
READY—对象响应状态(高为就绪)，$\overline{WR}$—MEM写或外设Out

思考：Intel 8088 CPU如何表示无操作、有访存操作？

$\overline{RD}\oplus\overline{WR}=0$

$(\overline{RD}\oplus\overline{WR})\cdot\overline{IO/\overline{M}}=1$

2、主存与CPU的连接　　（主存容量可配置的处理）

思考：CPU的地址传送是1次/2次？原因？　1次传送；最小化访存延迟，主存可为SRAM或DRAM

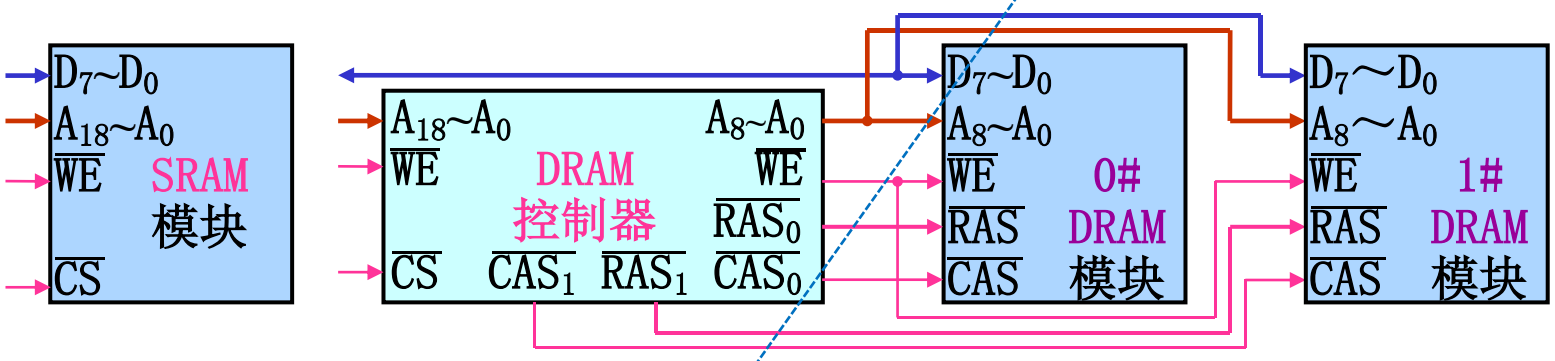
*主存的接口组织：

SRAM—数据引脚：　位数w＝CPU数据线位数　←CPU直接访问主存

地址引脚：　位数$k=\log_2$(主存单元个数)≤CPU地址线数

控制引脚：　片选＝1根，操作≥1根

DRAM— DRAMC的接口(同SRAM)　←字扩展用DRAMC实现



※提醒：后续仅讨论SRAM主存与CPU的连接　（简单易懂）

*主存在CPU可寻址空间中的位置：通常占用可寻址空间的低端

*主存与CPU的连接：

数据引脚—连接CPU数据线的全部 ←两者位数须相等

地址引脚—连接CPU地址线的低位 ←占用连续地址空间

└→地址线高位用于判断目标地址∈主存地址范围？

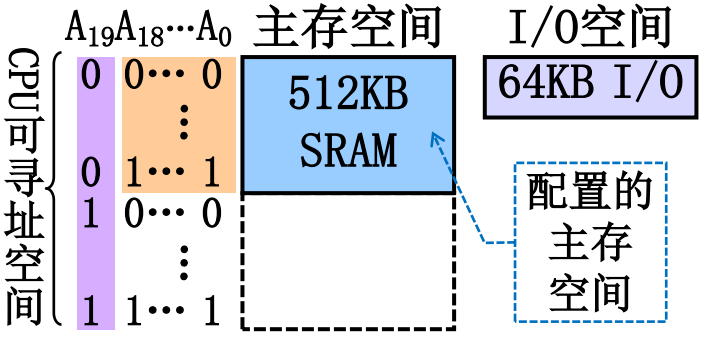
控制引脚—$\overline{CS}=\overline{(CPU有MEM操作)\cdot(目标地址\in主存地址范围)}$

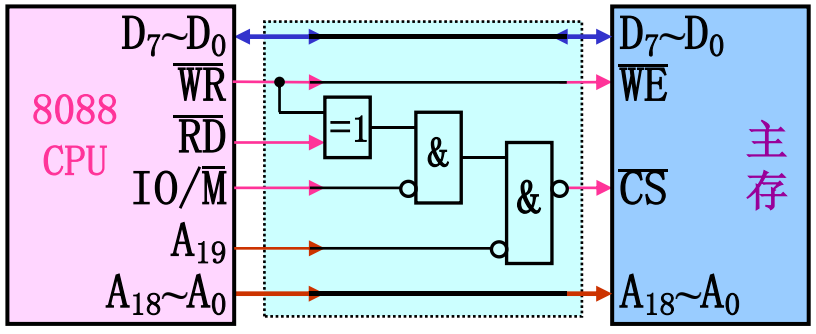
$\overline{WE}=\overline{(CPU操作类型为写)}$ ←WE#有效表示写操作

示例中：有操作的表示：$\overline{RD}\oplus\overline{WR}=1$，写操作的表示：$\overline{WR}=0$

访问部件为MEM的表示：$IO/\overline{M}=0$

目标地址∈主存地址范围的表示：$A_{19}=0$





例—某CPU有16根地址线、8根数据线，控制线为IO/$\overline{M}$、$\overline{RD}$、$\overline{WR}$。欲配置16KB主存(前4KB为只读空间)，可用芯片有2K×8b ROM及4K×4b SRAM。(1)主存由几片ROM及RAM组成？(2)写出主存中各芯片所占空间及其片选逻辑；(3)画出主存内部的连接图；(4)画出主存与CPU的连接图。

与CPU可寻址空间无关

←主存数据线为8位

解：(1)所需芯片数量—ROM＝2块，RAM＝6块

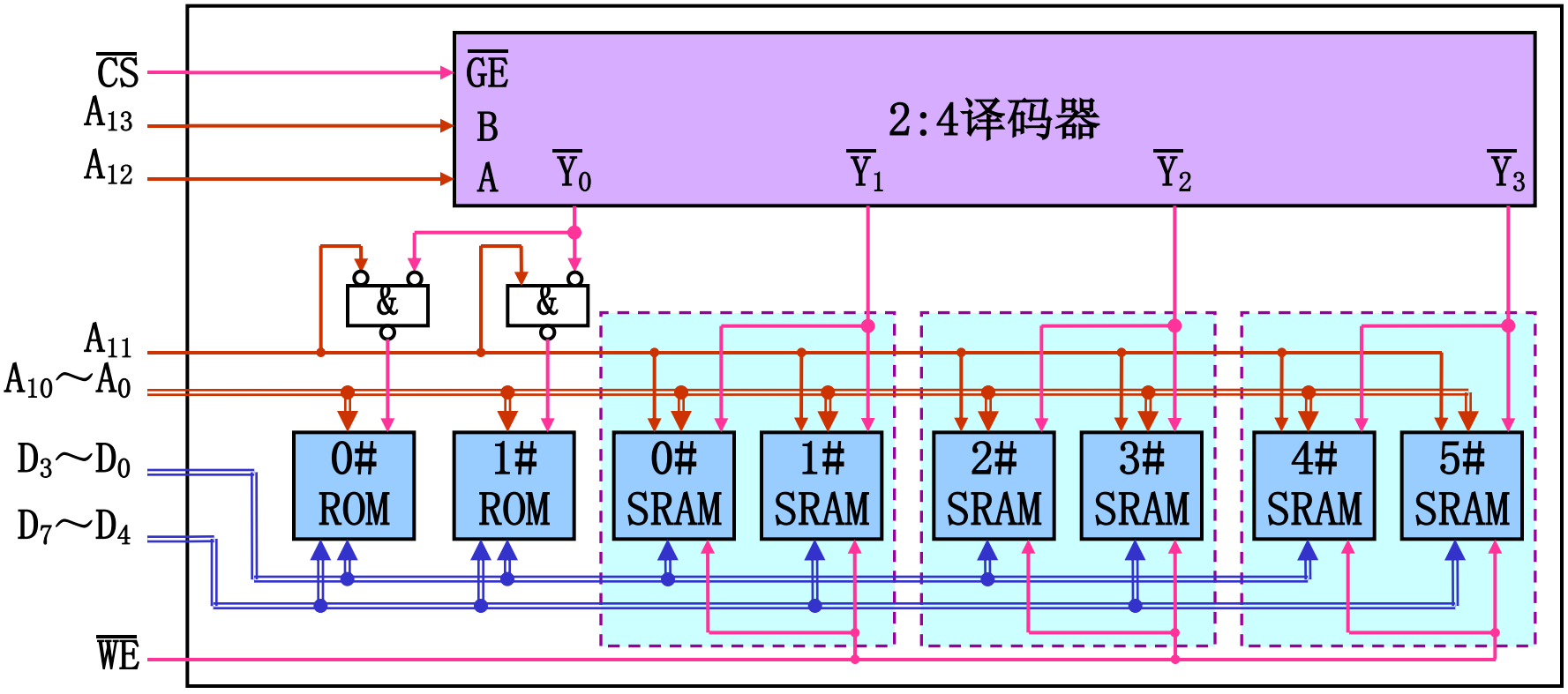
(2)芯片所占空间—地址/字内范围不同，主存地址线为14根

芯片片选逻辑— 主存被选中、引脚线地址∈芯片地址范围

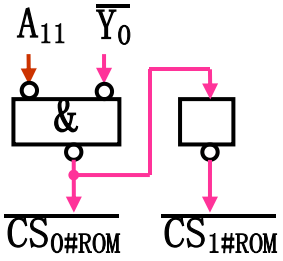
A_{13}~A_{10} A_9 ~ A_0	b_7 ~ b_4	b_3 ~ b_0
000X XXXXXXXXXX	0# ROM	
001X XXXXXXXXXX	1# ROM	
01XX XXXXXXXXXX	1# SRAM	0# SRAM
10XX XXXXXXXXXX	3# SRAM	2# SRAM
11XX XXXXXXXXXX	5# SRAM	4# SRAM

片选逻辑(地址部分)	
A_{13}~A_{12}=00	A_{11}=0
	A_{11}=1
A_{13}~A_{12}=01	
A_{13}~A_{12}=10	
A_{13}~A_{12}=11	

(3)主存内部的连接图—先位扩展、后字扩展



思考①：ROM $\overline{CS}$线的逻辑可以如右图吗？

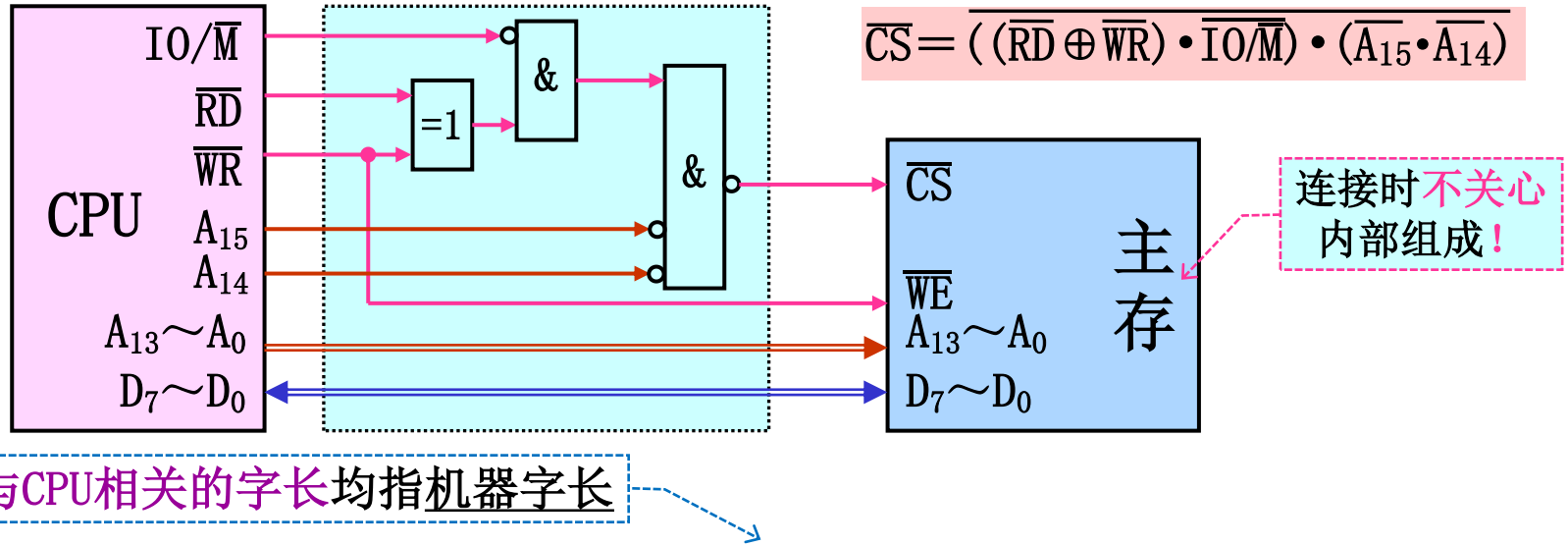


思考②：用3:8译码器实现时，各芯片的$\overline{CS}$线如何连接？

ROM — 第1片$\overline{CS}=\overline{Y_0}$，第2片$\overline{CS}=\overline{Y_1}$

SRAM— 第1组$\overline{CS}=\overline{\overline{Y_2}\oplus\overline{Y_3}}$，其余类似

(4)主存与CPU的连接图—假设主存地址从0开始



练习—某16位CPU中，存储器按字编址，可寻址空间为20位，数据引脚宽度等于存储单元长度。欲配置512KB主存(前128KB为只读空间)，可用的芯片有64K×8b ROM、64K×16b SRAM。画出主存中各芯片所占主存空间图、主存内部的芯片连接图，以及主存与CPU的连接图。

思考：主存-总线连接和主存-CPU连接，接线上有什么差别？

四、提高访存速度的技术 --主存的优化设计

*访存需求：一次访问多个连续单元，多次访存的地址连续
（数据长度＝x个单元） （程序访问局部性）

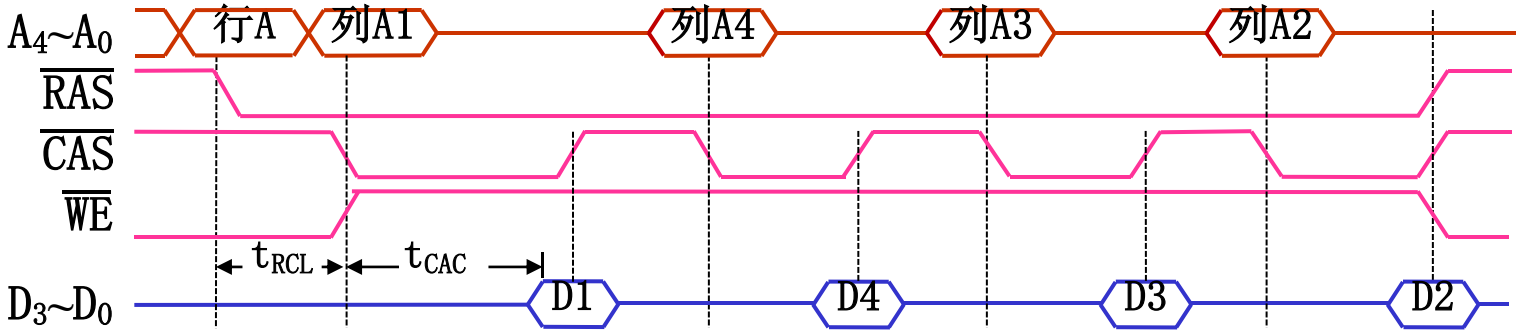
*优化方法：改进工艺、并行处理(多个MEM)、层次结构(多种MEM)

1、增强的DRAM --优化方法1(改进工艺或组成)

(1)FPM DRAM（Fast Page Mode DRAM）

└←页指存储矩阵中同一行的所有单元

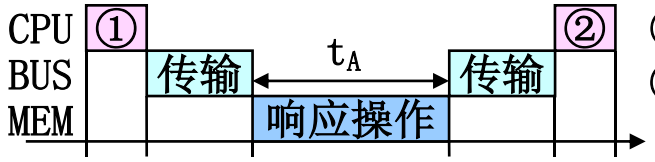
*优化原理：同一行的操作(列地址可不连续)可共用$\overline{RAS}$信号



*性能分析：$T_{DRAM}=n\cdot(t_{RCL}+t_{CAC}+t_{传送})$，$T_{FPM}=t_{RCL}+n\cdot(t_{CAC}'+t_{传送})$
仅列译码&I/O,<<t_{CAC}→┘

(2) SDRAM（Synchronous DRAM）

思考①：CPU读MEM有哪些环节？

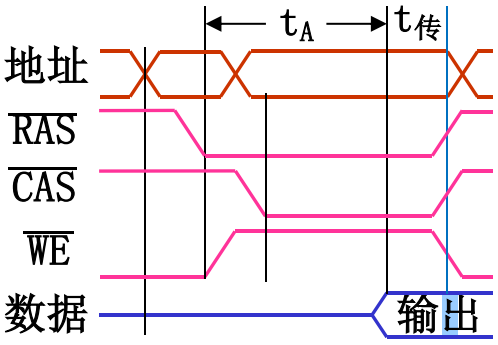


①发出地址/命令
②接收数据、
撤销命令/地址

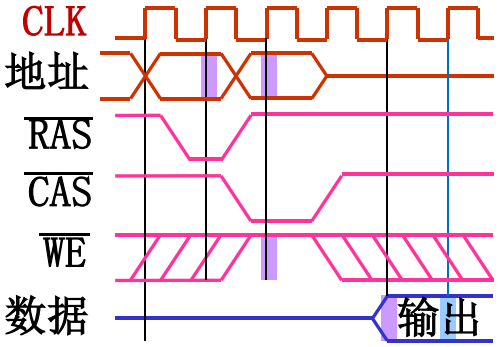
***优化原理1：采用同步工作方式(I/O与时钟同步)，可简化操作控制**

└→SDRAM锁存I/O信号→约定操作时延→┘

称为同步控制方式



异步控制(信号一直保持)



同步控制(信号无需保持)

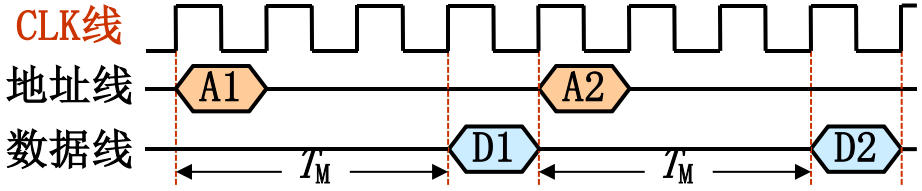
思考②：同步控制方式的好处？

操作者中途可离开(做其他工作)

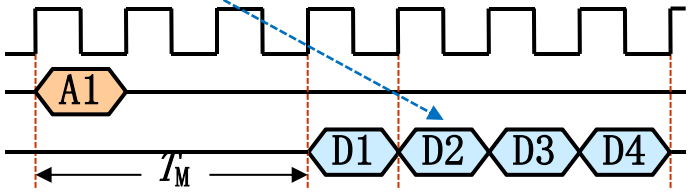
例：我在教室，请同学回去帮忙拿东西，期间我干什么？

***优化原理2：支持突发传输模式(多个地址连续数据)，可提高MEM带宽**

└→SDRAM用计数器改变列地址　←需同步工作方式支持

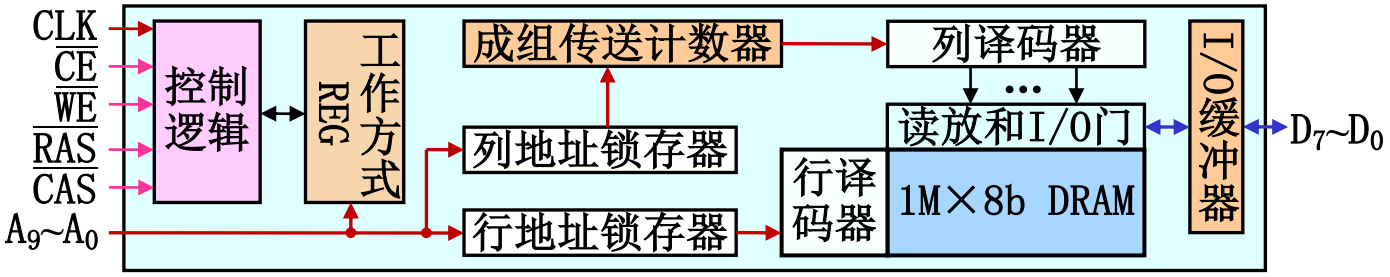


常规传输模式(带宽＝w/T_M)



突发传输模式(带宽＝$n \cdot w/T_M$)

*SDRAM组成示例：DRAM基础上，增设锁存器、计数器等



同步工作原理—I/O同步于CLK，内部操作异步

└→地址&数据需锁存

数据传送方式—仅支持突发传输方式，列地址用计数器改变

BL—Burst Length

└←常规传输时BL＝1，BL有几种

思考①：突发传输时数据间隔很短，为什么？

操作控制过程—

控制过程：①设置BL ②启动操作 ③传送数据 ←常规仅②③

过程优化：当前操作与上一操作BL相同时，缺省第①步

└←同一操作者自知

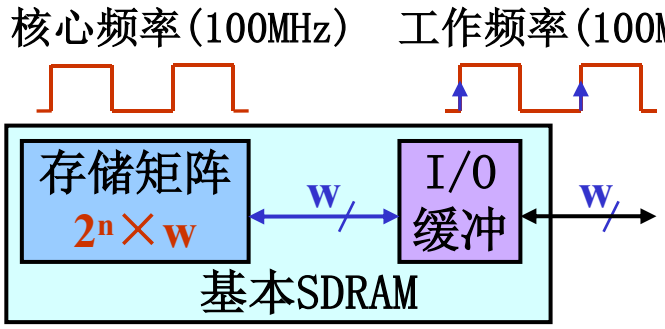
思考②：第①步缺省时，SDRAM如何知道当前的BL？ BL保存在工作方式REG中

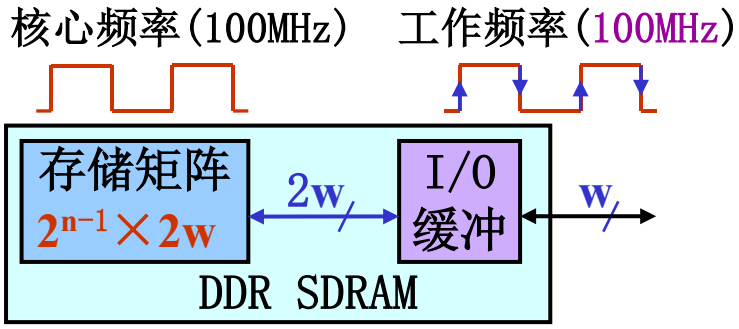
思考③：SDRAM必须支持突发传输模式吗？ 可不支持！无需计数器，无需设置BL

(3)DDR SDRAM（Double Data Rate SDRAM）

*优化原理： CLK的上升沿及下降沿都I/O ←SDRAM仅上升沿

*DDR SDRAM组成： 存储矩阵中单元宽度是I/O宽度的2倍，I/O时需拆分/合并；工作频率＝核心频率

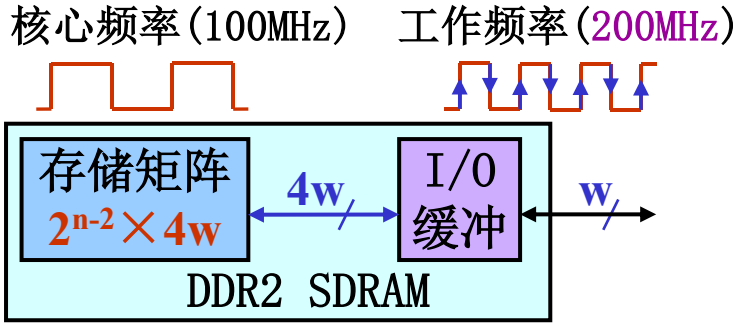




*DDR2 SDRAM组成：

存储矩阵中单元宽度是I/O宽度的4倍，I/O时需拆分/合并；

工作频率＝核心频率×2



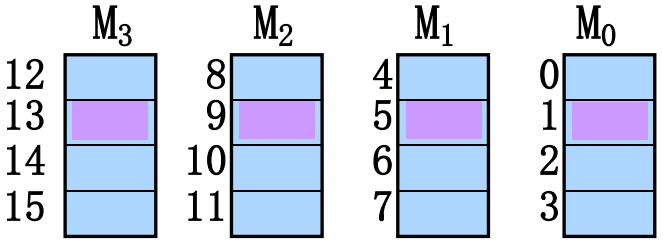
思考：DDR3 SDRA的内部结构？

2、多体交叉存储器 --优化方法2(并行处理[多个MEM])

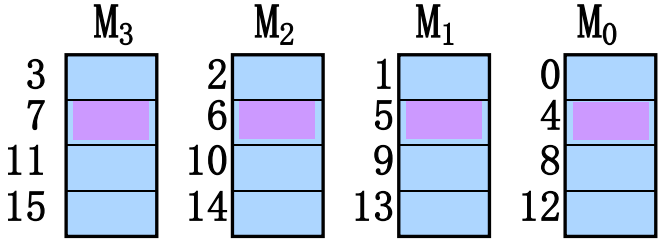
存储体—指具有存储矩阵、译码电路、读写电路的存储模块

访存需求：一次访问多个连续单元，多次访存地址连续

*多体MEM的编址方式：



(a)顺序编址方式



(b)交叉编址方式

思考：交叉编址方式实现时，MEM的内部连线与顺序编址有何不同？

应用选择— 容量扩展时，采用顺序编址；

性能优化时，采用交叉编址(各体同一单元地址连续)

*交叉编址MEM工作方式： 交叉访问方式、并行访问方式

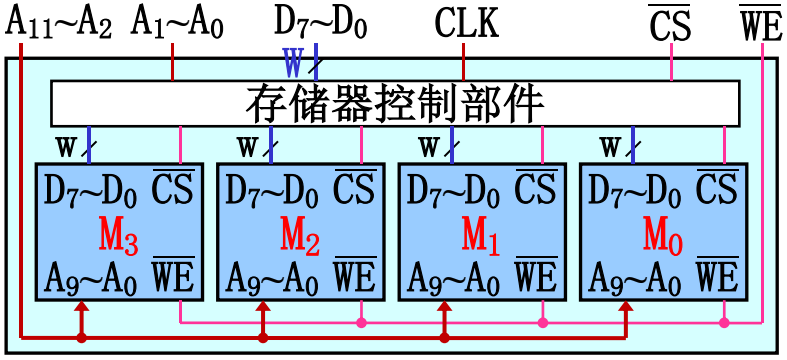
(不同存储体的访问方式) └←称为多体交叉MEM └←有称多体并行MEM

(1)交叉访问方式 (称为多体交叉MEM)

*优化思路：轮流访问(有重叠)各个体(≤m个)，各个体分时I/O(突发传输)

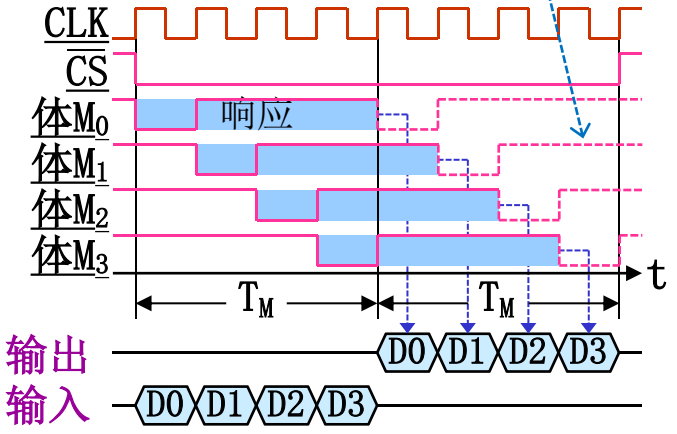
└←$T_{启动间隔}=T_M/m$　　└←数据引脚W＝w

*MEM结构：采用同步RAM接口，首个体号用地址低位表示



思考：MEM如何知道突发个数？

某些MEM支持突发长度＞m



*工作原理：　(同步工作方式)

每隔T_M/m启动一个体(T_M后即可I/O)

I/O采用突发传输方式(间隔＝启动)

*性能分析：$T_M'=(2-1/m)\times T_M$，$B_M'=m\times B_M$

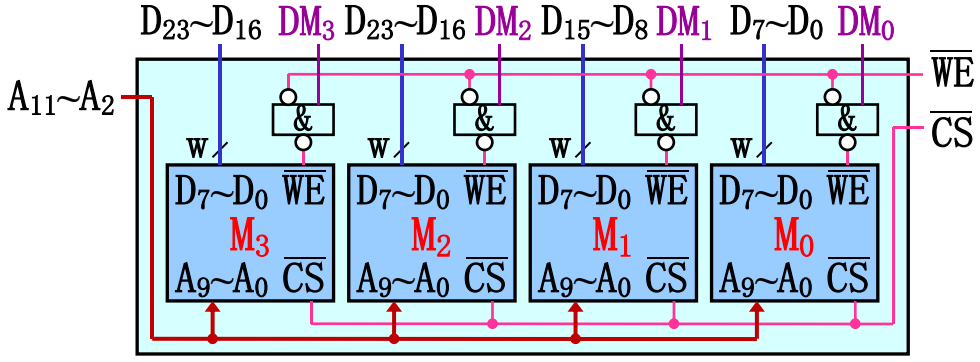
*与CPU的连接：与同步RAM相同

(2)并行访问方式 (有称多体并行MEM)

*优化原理：同时访问各个体(≤m个)，各个体同时I/O(常规传输)

└←$T_M' = T_M$　　└←数据引脚W＝m×w

*MEM结构：增设数据掩码引脚(指明需写入体号)　←读出体号处理放在CPU中



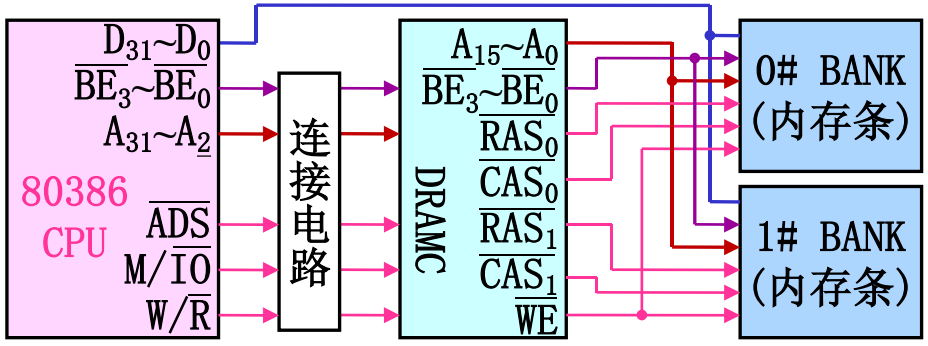
DM_3~DM_0	读长度	写长度
1111	4B	4B(M_0起)
0011		2B(M_0起)
1100		2B(M_2起)
某位为1		1B(M_i)

思考：为何没有地址低位？

*工作原理：同时启动各个体，I/O采用常规传输方式($B_M' = 4 \times B_M$)

*与CPU的连接：CPU—数据宽度＝m×w，有数据掩码、无地址低位引脚

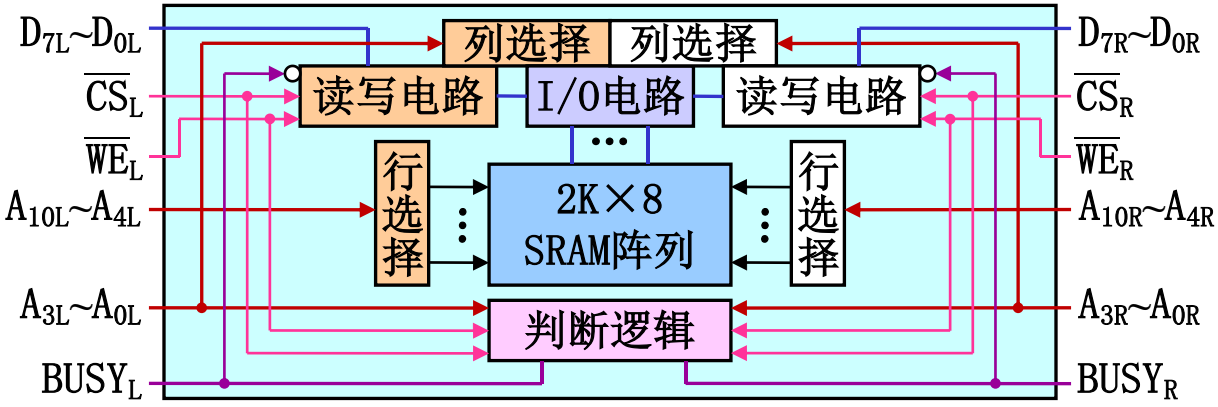
DRAMC—默认地址低位为0，其余同DRAM



3、双端口MEM　　--优化方法2(并行处理[多个操作并行])

*优化原理：同时处理2个访问请求，即W′＝w＋w，$T'_M = T_M$

*MEM结构：增设译码及读写电路、冲突判断电路



存储元的连接—有2个字选择线

冲突检测逻辑—$A_{列L} = A_{列R}$时有冲突，使某个BUSY有效

*应用：常用于Cache(查找与更新)

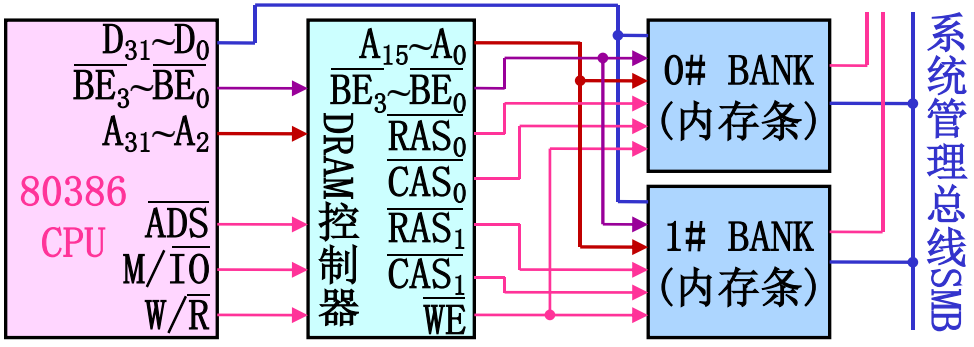
作业3-2：P137—13、17、20

引子—不改变连接电路时，如何实现主存容量的可配置？

※现代计算机的主存子系统组成：（不作要求）

①DRAMC支持有限数量的主存模块 ←DRAM字扩展方法所致

（≤BANK插槽数）（内存条） ←容量可选配所致







②主存模块的接口统一、容量可变 ←提高灵活性、容量可配置

└→早期168线DIMM 、目前288线DIMM，均支持多体并行MEM方式

③系统启动时，BIOS检测内存条容量、分配内存条地址范围

硬件支持— 内存条包含SPD芯片、连接SMB

SPD—Serial Presence Detect

SMB—System Management Bus

容量检测— 依次读取各BANK中内存条的容量

└←轮流选中SPD芯片

写入DRAMC相应REG中

地址分配— 按BANK顺序约定各内存条的地址范围

④系统工作时，DRAMC根据所接收地址选择内存条(使RAS_i有效)、中转操作

引子—不满意主存速度时，可采用优化方法3(多种MEM[层次结构])

§3.4 高速缓冲存储器

※主要内容：Cache的组成、工作流程、实现技术(映射/替换/写)

*MEM的层次结构回顾：

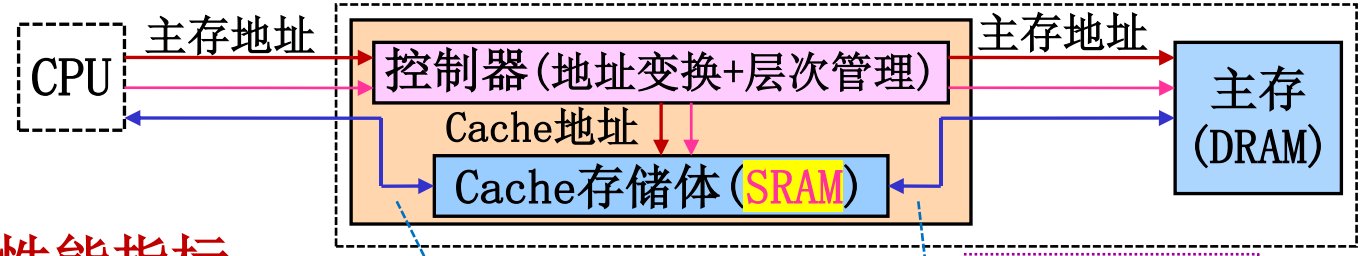
多种MEM按速度分层(级联)，各层之间协调工作(层间透明地传递信息)

└←$S_{M(i)}<<S_{M(i+1)}$、$T_{M(i)}<<T_{M(i+1)}$　　└←M_i中信息为M_{i+1}的副本(改后需写回M_{i+1})

Cache-主存层次—提高访存速度(地址变换&层间管理由硬件实现)

一、Cache的基本原理

*Cache的外部接口：　由SRAM组成，按主存地址访问



*Cache的性能指标：

层间级联所致

命中率—$H=N_{Cache}/(N_{Cache}+N_{MM})$＝命中Cache次数/访存总次数

平均访问时间—$T_A=H\cdot T_{Cache}+(1-H)(T_{Cache}+T_{MM})=T_{Cache}+(1-H)T_{MM}$

$=T_{命中}+(1-H)T_{缺失}$　　←$T_{缺失}$指停顿时间(如调入/替换)

1、Cache的存储空间管理

***与主存的信息交换单位：** —目标是减小T_A，$T_A=T_{命中}+(1-H)T_{缺失}$

分析—相邻信息一起放入Cache，采用突发传输方式

└←可提高H←利用局部性　　└←可减少$T_{缺失}$←$T_{突发}=T_{存取}+nT_{传送}$

结果—为块(Block)，块大小＝n个主存字(主存单元中内容)，n为常数

***存储空间管理：** —面向CPU的访问和与主存交换

编址单位— 同主存(Cache仅为主存的缓冲器)　　←面向CPU的访问

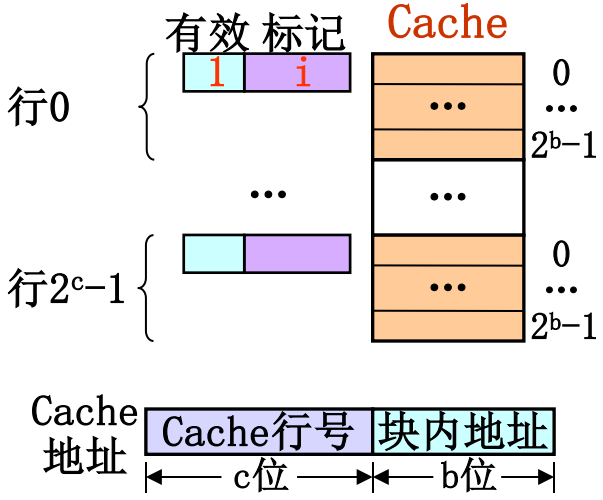
交换管理—①主存与Cache的空间，都按块大小划分成若干区域

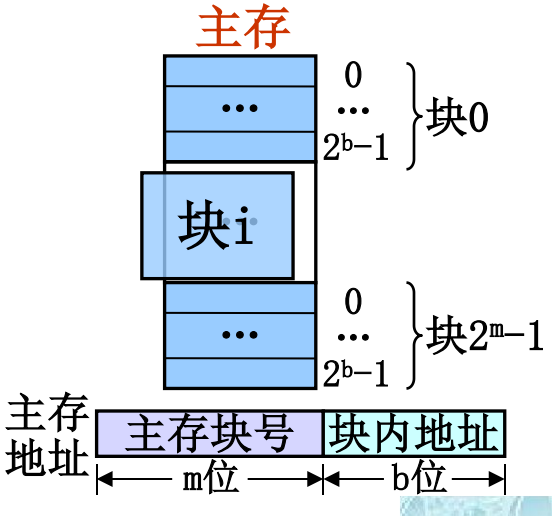
②Cache每行设置有效位(V)、标记(Tag)保存元件

表示行中是/否有数据→┘　　└←表示数据来自哪个主存块

思考①：主存块内的2#单元，在Cache行中会是第几号单元？

思考②：若CPU访存地址为a#块的b#单元，Cache怎知其是/否在行j中？

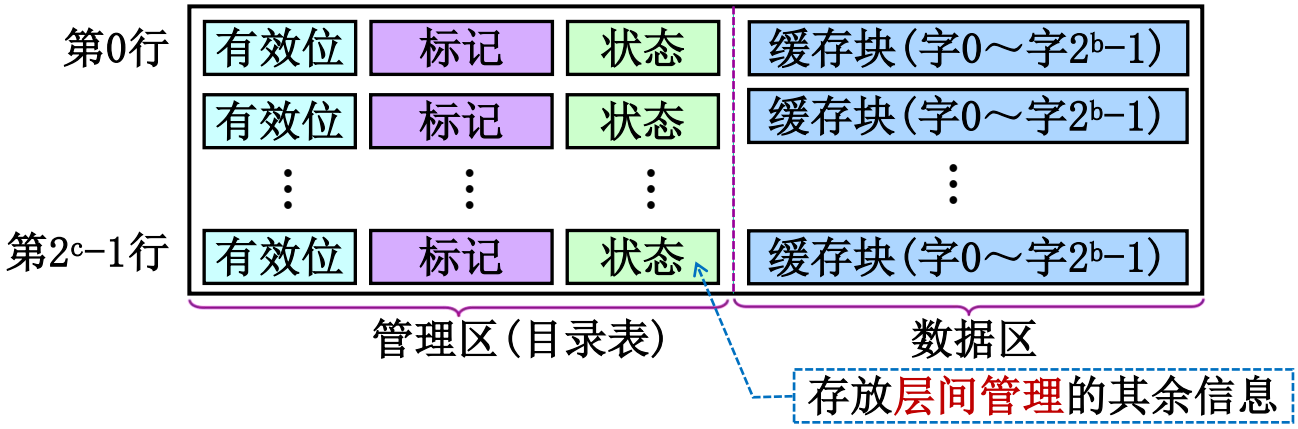




*Cache结构抽象：

①为行的数组，每行包含缓存块、管理信息

└←大小&编址单位均都与主存块相同



Cache容量—指Cache数据区的容量

Cache总容量—指Cache数据区＋管理区的容量之和

示例：主存按字节编址、地址空间为1M，主存块为16B，Cache容量为8KB。Cache有多少行？行号多少位？块内地址多少位？

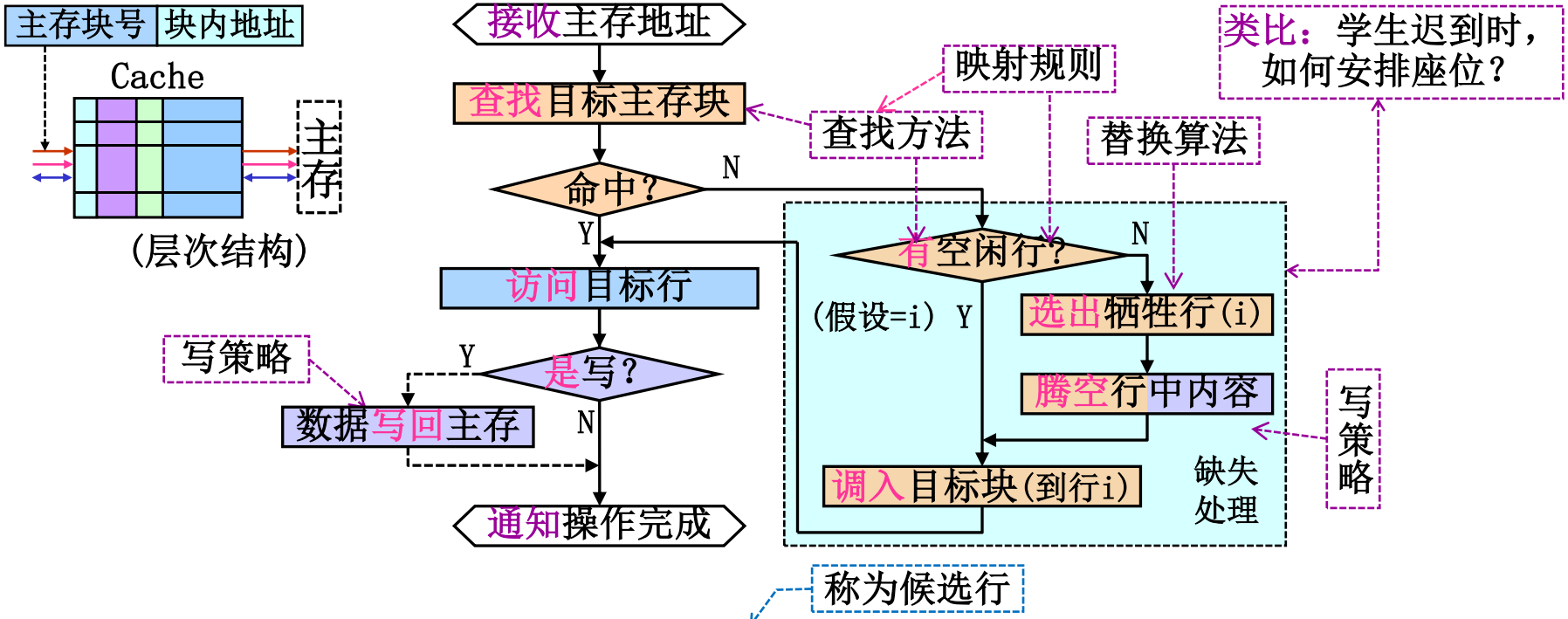
②控制器为操作处理机构(查找/替换/读写)，处理访问请求

└←实现地址变换+层间管理

2、Cache的工作原理

***操作过程：** ①地址变换，②数据访问，③数据写回主存

（查找目标行） （访问行中数据） （保持层次一致性）



***实现技术：** 映射规则(块可调入哪些行)， 查找方法(如何找到目标行)

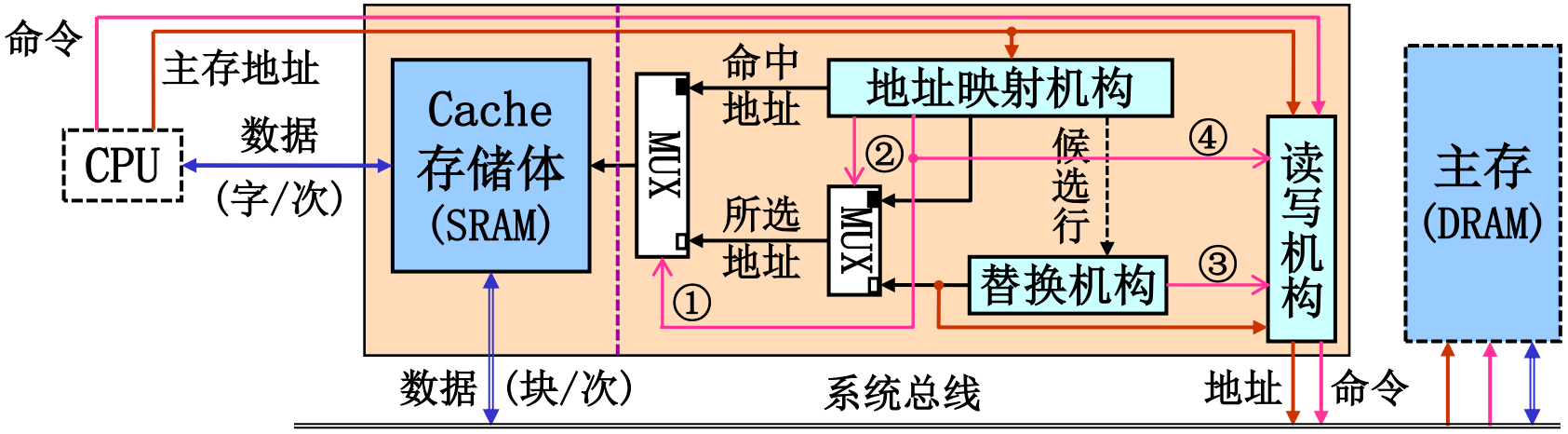
常放在一起处理

在候选行中找

替换算法(如何选择牺牲行)， 写策略(何时/如何写回主存)

3、Cache的基本结构

***基本组成：** 存储体＋控制器(映射[含查找]/替换/读写机构)



图注：①是/否命中　②是/否有空行　③是/否需写回　④是否需调入

■、□表示MUX控制端信号为1、0所选择的入端

操作状态—

命中			①=1、　③无效、④无效
缺失(需调入)	有空闲行		①=0、②=1、③无效、④有效
	无空闲行(需替换)	牺牲块未改过	①=0、②=0、③无效、④有效
		牺牲块已改过	①=0、②=0、③有效、④有效

***控制器组成：** (相关环节中讨论)

二、Cache的地址映射

*任务：确定一个主存块可放到Cache的哪些行，如何查找该主存块

称为候选行←┴→取决于映射方式

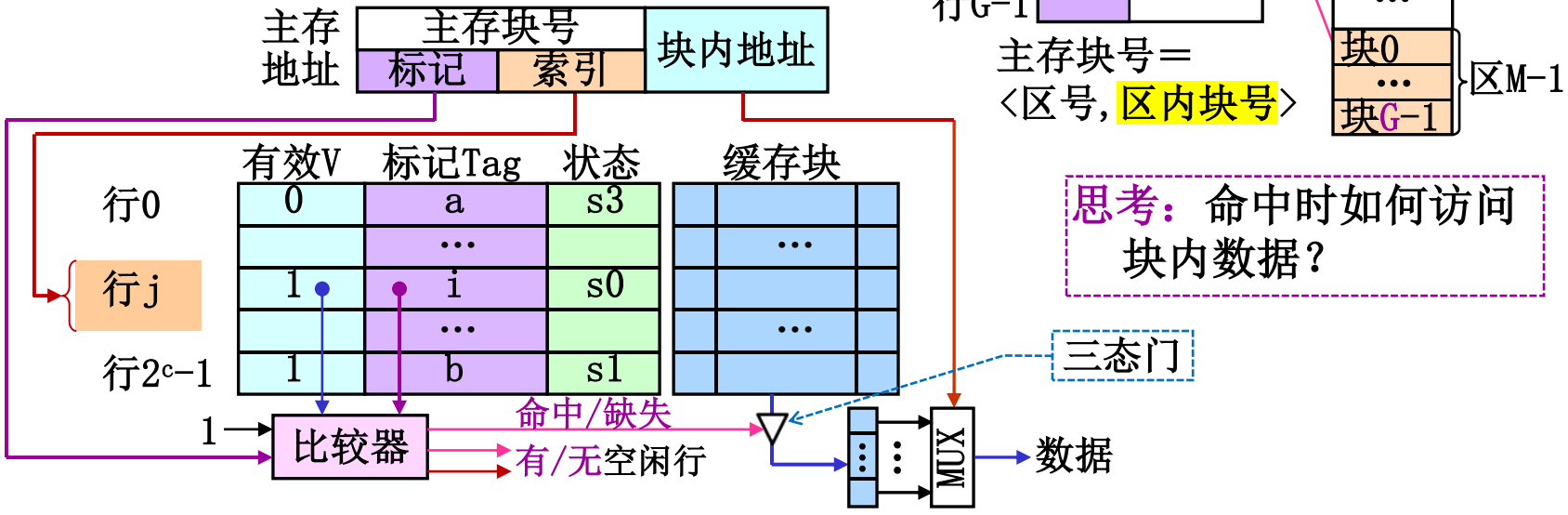
*映射方式：直接(1行)、全相联(所有行)、组相联(n行) ←候选行数不同

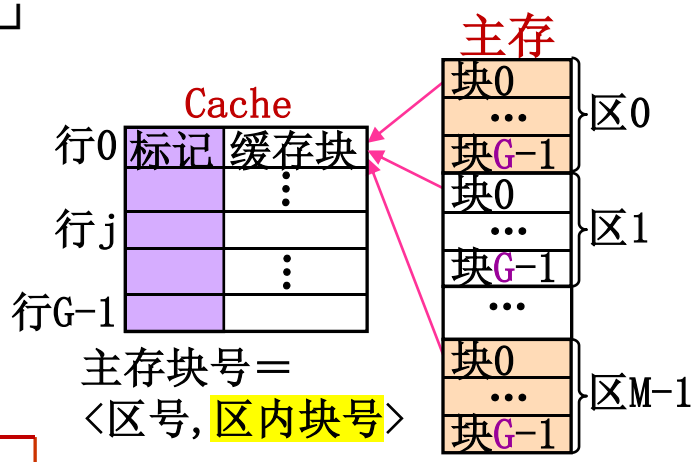
└→候选行地址由主存块号决定←┘

*查找组织：

①划分地址—主存块号＝<标记, 索引>

②比较标记—在候选行(索引指定)中比较





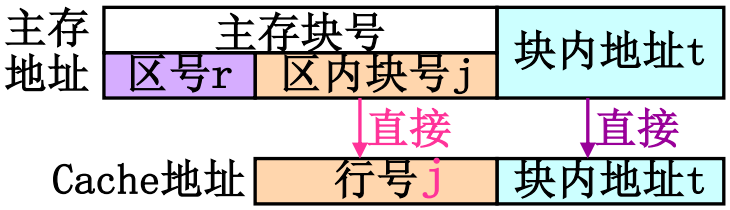
思考：命中时如何访问块内数据？

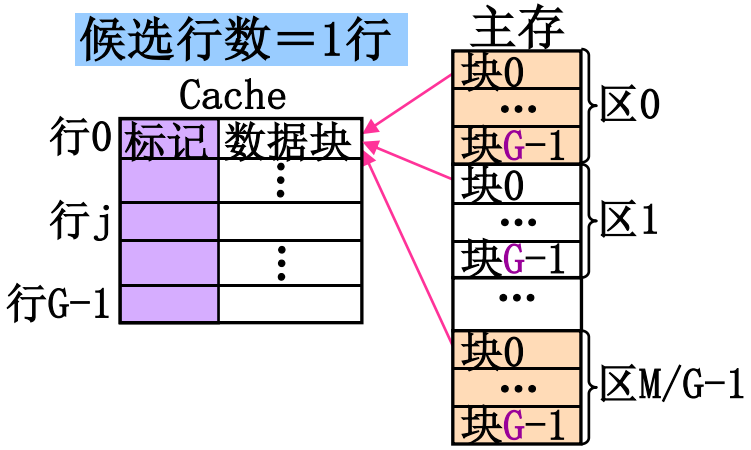
*性能指标：块调入的冲突率(≠命中率[访问时])，查找的速度与成本

1、直接映射(Direct mapping)

*映射规则：主存块i可放到Cache的某个行j中，j＝i mod G

*地址划分：





索引＝区内块号(=$\log_2 G$位)

标记＝区号(=$\log_2 M-\log_2 G$位)

例1：主存按字节编址、地址空间为1M，主存块大小16B，Cache容量为8KB。采用直接映射方式时，(1)Cache行的标记(Tag)为几位？(2)访存地址为36454H时，可能命中的Cache行号？命中时目标行的V(有效位)、Tag的值？

解：(1)主存地址为$\log_2(1M)$＝20 位，块内地址为$\log_2(16B/1B)$＝4 位，

Cache行号为$\log_2(8KB/16B)$＝9 位，标记＝20-4-9＝7位；

(2)访存地址36454H＝ 0011 0110 0100 0101 0100 B，

可能命中的Cache行号＝ 0 0100 0101B＝045H，

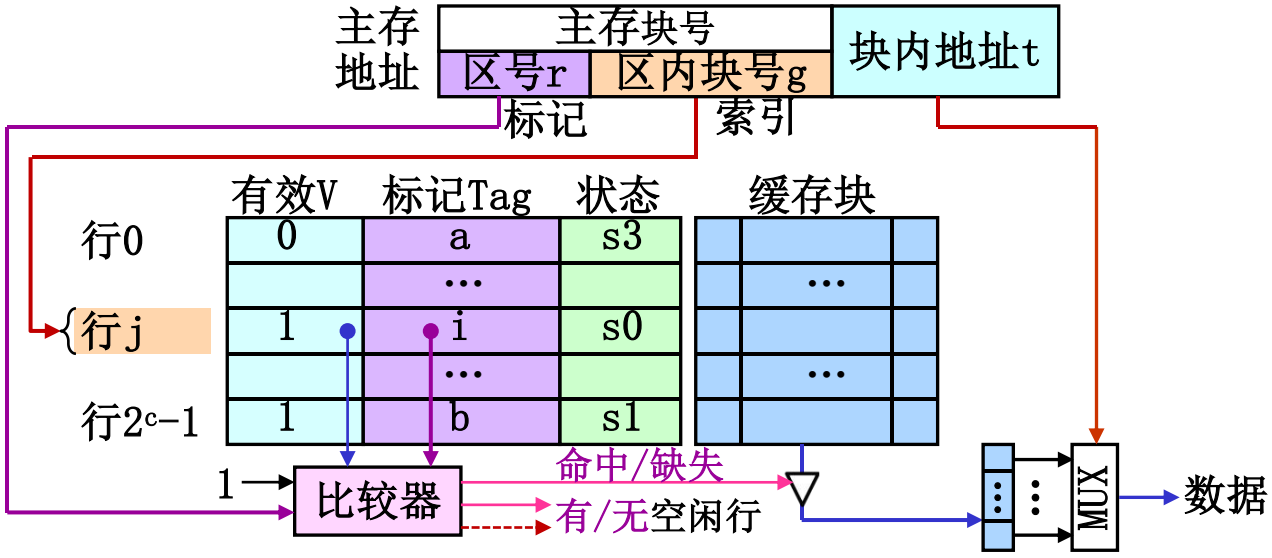
命中时目标行的V＝ 1 、Tag＝0011 011B＝1BH。

*地址变换：（候选行只有1行）

查找方法—用1个比较器进行比较

查找处理—命中条件为候选行的V＝1、Tag＝主存地址中区号；

有空闲行条件为候选行的V＝0，空闲行＝候选行



数据访问—变换时读缓存块(仅1个)，变换后读/写存储单元

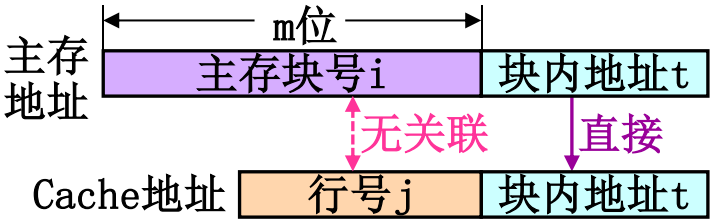
*映射性能：冲突率最高，地址变换速度最快、成本最低

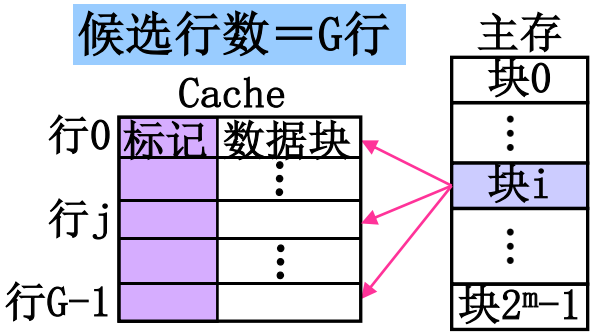
└←思考：为什么？怎么优化？

2、全相联映射(Fully associate mapping)

*映射规则：主存块i可放到Cache的任意行j，$j \in \{0, 1, \cdots, G-1\}$

*地址划分：



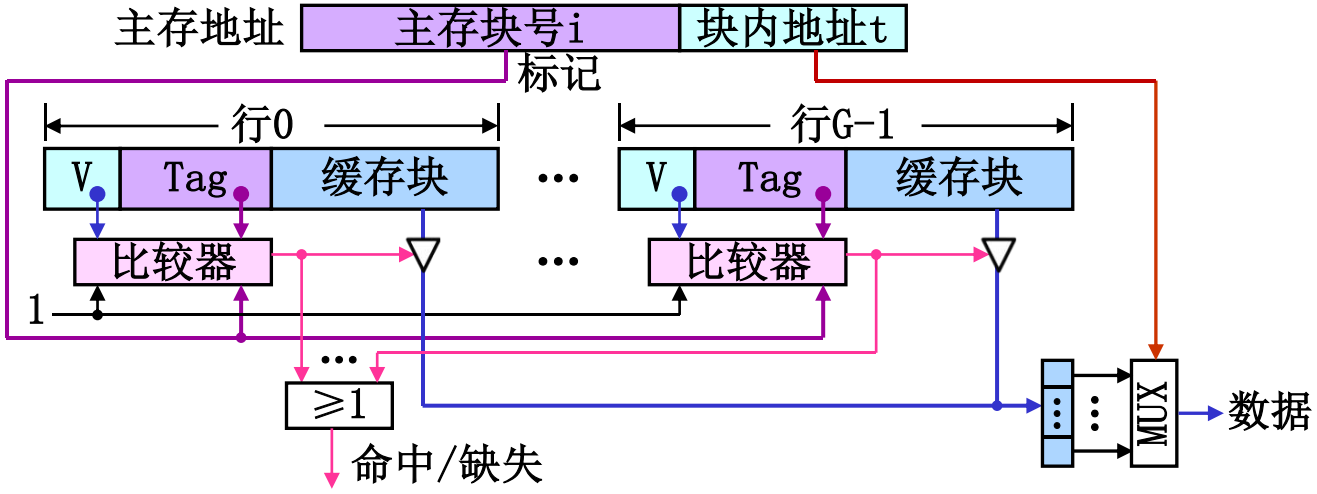


索引＝无(=0位)，标记＝主存块号(=m位)

*地址变换：(候选行有G个[=Cache行数])

查找方法—用G个比较器同时比较 ←$T_{比较}=\Delta t$，肉疼；串行比较太慢

查找处理—



思考①：可能有多行命中吗？

思考②：有空闲行的条件？

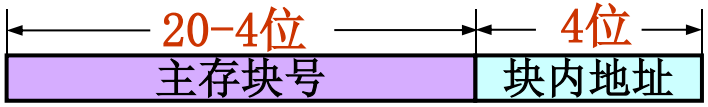
数据访问—

*映射性能：冲突率最低，地址变换速度较快、成本最高

例2：同例1（存储器按字节编址，CPU可寻址空间为1M，主存块大小为16B，Cache数据区容量为8KB）。采用全相联映射方式时，(1)主存地址如何划分？(2)Cache行的标记(Tag)为几位？(3)CPU访存地址为36454H时，则命中时目标行的V(有效位)、Tag的值？

①存储器编址单位＝主存编址单位＝程序MEM编址单位
②CPU可寻址空间＝主存地址空间
③Cache数据区容量＝Cache容量

解：(1)主存地址(20位)划分如下：　(块内地址为4位)



(2)Cache行的标记＝主存块号，Tag为16位

(3)访存地址36454H＝ 0011 0110 0100 0101 0100 B，

目标行的V＝1 、Tag＝ 3645H

3、组相联映射方式(Set associate mapping)

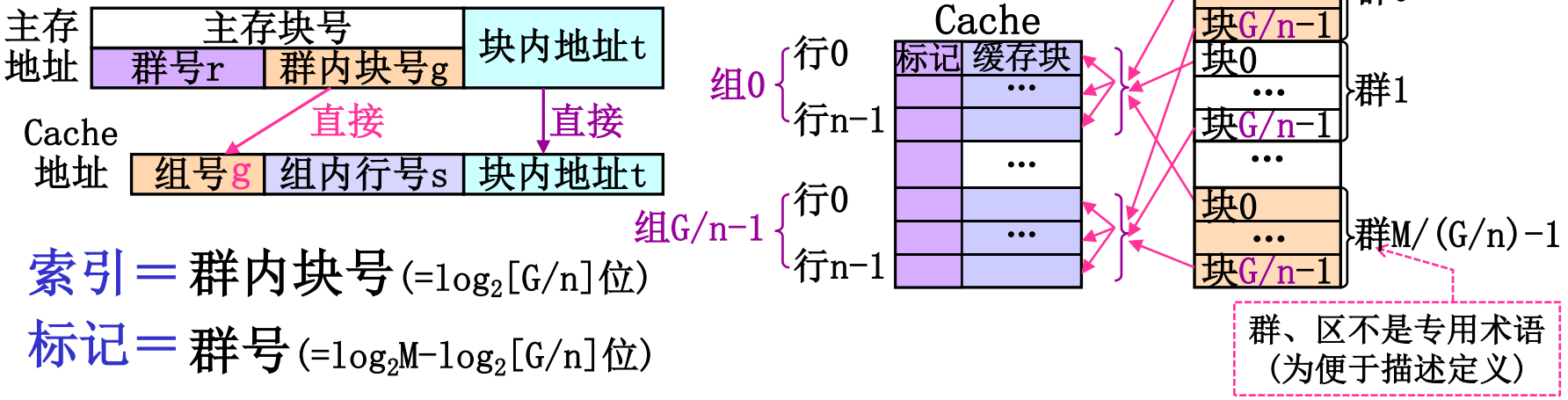
*映射规则：将Cache的行分组，每个组有n个行；→ 称为n路组相联

主存块i可放到Cache某个组j的任意行，j＝i mod G/n

块→组(直接)、组→行(全相联)

候选行数＝n行

*地址划分：



索引＝群内块号(=$\log_2[G/n]$位)

标记＝群号(=$\log_2 M-\log_2[G/n]$位)

例3：同例1(主存按字节编址、地址为20位，块大小为16B，Cache容量为8KB)。4路组相联映射时，标记为几位？访存地址为36454H时，命中时的组号、目标行的Tag值？

解：Tag＝群号＝20-log2(16B/1B)-log2((8KB/16B)/4)＝20-4-7＝9位

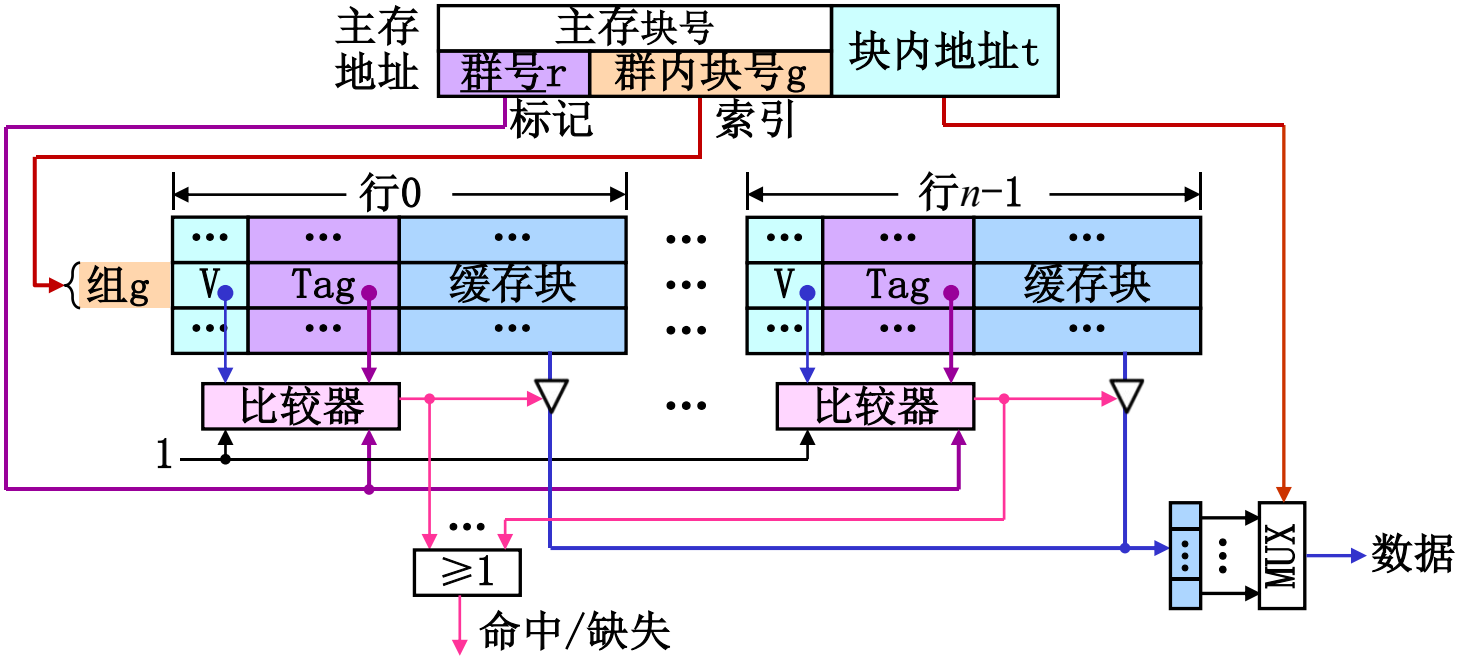
36454H＝<001101100,1000101,0100>，命中时组号＝45H、Tag＝06CH

*地址变换：（候选行有n个[=组内行数]）

查找方法—用n个比较器同时比较 ←$T_{比较}=\Delta t$，成本尚好

查找处理—命中条件为某候选行的V＝1、Tag＝主存地址中群号；

有空闲行条件为某候选行的V＝0，空闲行＝该候选行



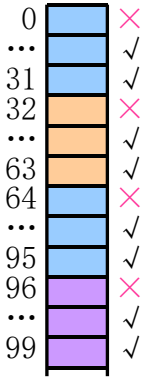
*映射性能：冲突率较低，地址变换速度较快、成本较低

└←n通常≤8，命中率≈全相联方式

例4：主存按字节编址、地址空间为24位；Cache容量为8KB，块大小32B，采用4路组相联映射方式。⑴主存地址如何划分？ ⑵若Cache初态为空，CPU从0#单元起连续读出100B数据(1B/次)，Cache命中率是多少？ ⑶若⑵中100B数据从62#单元起存储，Cache命中率是多少？ ⑷若⑵中$T_{命中}$=20ns、$T_{缺失}$=200ns，平均访问时间是多少？

解：⑴主存地址(24位)划分如下： (块内地址为5位)





⑵数据放在连续的 $\lceil(100+0\%32)/32\rceil=4$ 个主存块中，

块调入Cache时，放在 0#～3# 组中，有 0 次冲突；

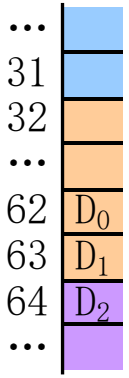
访问时不命中的特征是 每个块的块内地址＝0时缺失

Cache命中率＝ (100-4)/100＝96%

⑶数据放在 $\lceil(100+62\%32)/32\rceil=5$ 个块中，H＝95%

⑷$T_A=T_{命中}+(1-H)T_{缺失}=20+0.04*200=28ns$

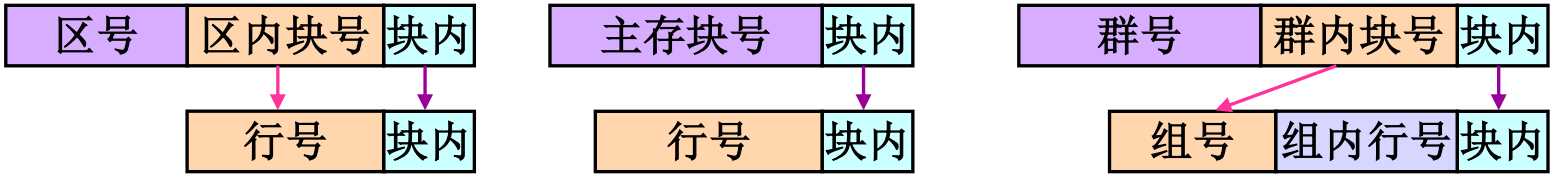
思考：为何$T_A\neq0.96*20+0.04*200$？



※地址映射小结：

设主存块号为m位，Cache有G行($G=2^c$)，主存块i调入Cache行j

	直接映射	全相联映射	组相联映射
候选行数	1	G	n
映射规则	j＝i mod G	j∈{0,1,…,G-1} (与i无关)	j＝<k,x>，k＝i mod G/n x＝0～n-1
索引位数	$\log_2$行号＝c	0(无索引)	$\log_2$组号＝c-$\log_2$n
标记位数	$\log_2$区号＝m-c	$\log_2$主存块号＝m	$\log_2$群号＝m-(c-$\log_2$n)



映射方式抽象—均为组相联映射方式，路数、组数不同

路 数	1	G	n
组 数	G	1	G/n

相联度—指一个主存块可映射到的Cache行数，即候选行数(路数)

查找的组织—使用n个(n=相联度)比较器同时比较 ←相联查找

应用选择—常为组相联方式

思考：4路组相联时比较器个数？

三、Cache的替换算法

*任务：从候选行中选出1个牺牲块(行)　←缺失&无空闲行时

└→适于全相联/组相联映射　←直接映射没得选

*性能指标：对命中率H的影响程度、算法的实现开销

└←替换是/否遵循访问局部性，如增加相联度时

最佳算法—选择最远将来使用的块为牺牲块　←无法实现(没长后眼)

*常见替换算法：—基于候选行

RAND(随机)—牺牲块为随机的块　←H随机,适于全相联

FIFO(先进先出)—牺牲块为最早调入的块　←H随机(≠访问次序)

LRU(最近最少使用)—牺牲块为近期最少使用的块　←H随相联度增大

*LRU算法的实现：　思考：4路组相联映射时，每个行的CNT为几位？

各块次序的表示—各行用计数器CNT存放，值越大表示越久未用

第i组 | V | Tag | LRU | 缓存块 | … | V | Tag | LRU | 缓存块 | LRU＝$\log_2 n$位

各块次序的更新—访问时，目标块CNT←0、其余块CNT++(饱和计数)

└←块次序不变(目标块已=0)时均不变

牺牲块的选择—选择CNT最大的块

例1：全相联Cache有4行，采用LRU替换算法，Cache初态为空，CPU按下列块地址流访存时，分别求每个块只访问1次、连续访问4次时的命中率H

状态 / 行号	V	标记	CNT	V	标记	CNT	V	标记	CNT	V	标记	CNT	V	标记	CNT	V	标记	CNT	V	标记	CNT	V	标记	CNT
行0	0	*	3	1	2	0	1	2	1	1	2	0	1	2	1	1	2	2	1	2	3	1	5	3
行1	0	*	3	0	*	3	1	11	0	1	11	1	1	11	2	1	11	3	1	6	0	1	6	1
行2	0	*	3	0	*	3	0	*	3	0	*	3	1	9	0	1	9	1	1	9	2	1	9	0
行3	0	*	3	0	*	3	0	*	3	0	*	3	0	*	3	1	7	0	1	7	1	1	7	2
块地址	初始			2			11			2			9			7			6			9		
操作状态	第1次			调入			调入			命中			调入			调入			替换			命中		
	2~4次			命中			命中			命中			命中			命中			命中			命中		

次序不变($CNT_{目标块}=0$)时，各CNT不变；CNT=3时不再增加(饱和计数)

每个块只访问1次时，H＝(7-5)/7＝2/7；连续访问4次时，H＝23/28

思考：增加相联度，LRU算法能提高H吗？

四、Cache的写策略

*任务：确定写操作的数据何时写回主存　　（又称更新策略）

思考①：Cache中数据为何要写回主存？　多个CPU共享主存、设备直接访问主存时

*性能指标：对T_A、总线的影响程度　　[$T_A = T_{命中} + (1-H)T_{缺失}$]

*写策略类型：（按写命中来命名）

写命中时—全写法：同时写Cache和主存(1个字)　←可保持一致性

写回法：只写Cache，替换时写主存(整个块)　←可优化T_A

思考②：替换时块没改过也要写主存吗？　└←块被改过时才写

写缺失时—写分配法：目标块调入主存，再按写命中处理

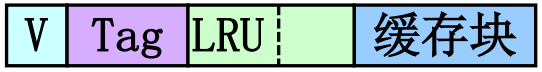
非写分配法：不调入目标块，直接写主存

写命中&缺失的组合—全写法＋非写分配法，写回法＋写分配法

（反正要写主存）　（遵循访问局部性）

*包含写策略的Cache行组成：

全写法的行组成：



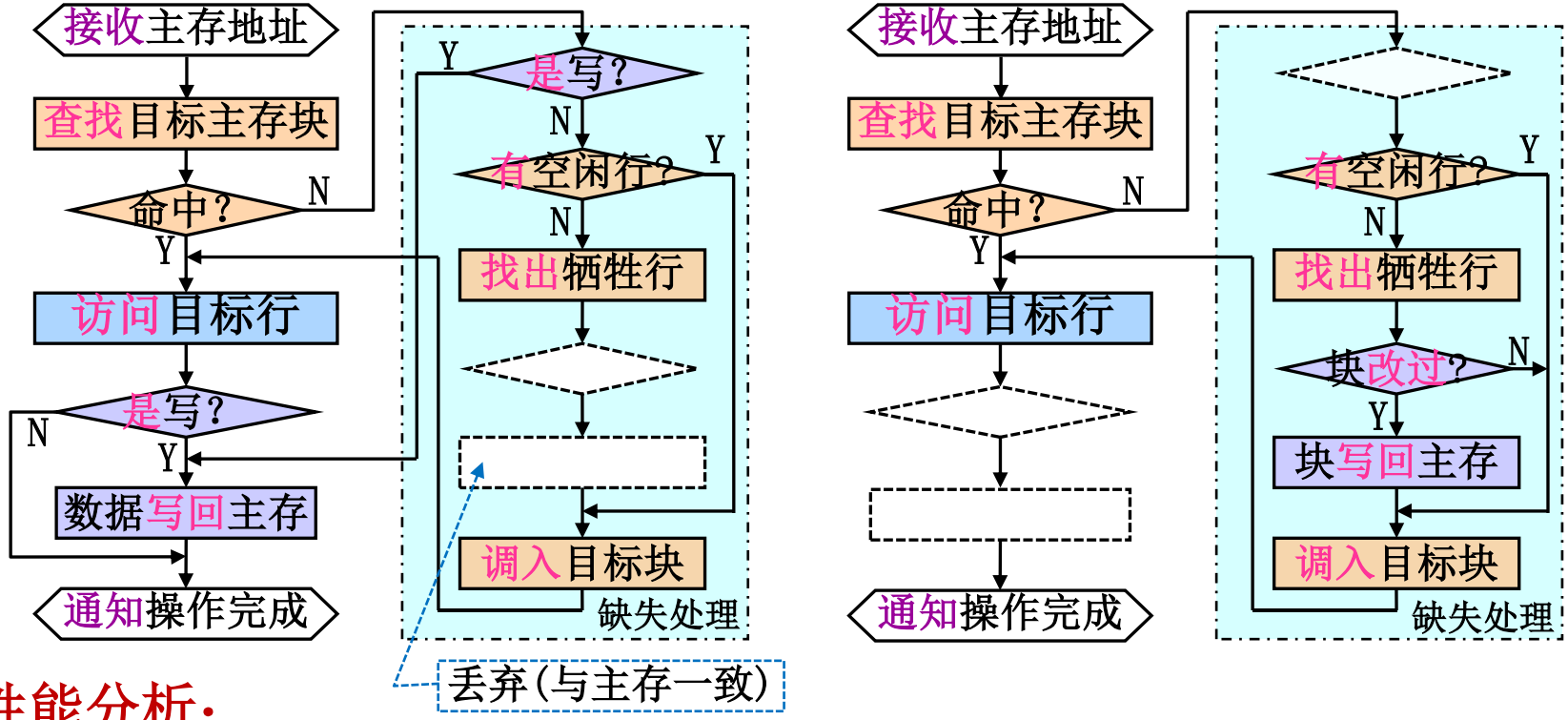
写回法的行组成：



称为脏位(表示块是否改写过)

全写—Write Through
写回—Write Back
写分配—Write Allocate
非写分配—Not Write Allocate

*操作过程：



*性能分析：

	$T_{读命中}$	$T_{读缺失}$	$T_{写命中}$	$T_{缺失(写)}$	总线占用
全写法	$=T_{Cache}$	$=T_{Mem(块)}$	$=T_{Mem(字)}$①	0②	读缺失、每次写
写回法	$=T_{Cache}$	$\geqslant T_{Mem(块)}$③	$=T_{Cache}$	$\geqslant T_{Mem(块)}$	读缺失、替换时
注：①>>T_{Cache}；②$T_{缺失}$指$T_{停顿}$(即$T_A-T_{命中}$)；③替换时＞、仅调入时＝					

*写策略应用：常采用写回法($T_{命中}$小、占用总线少)

五、Cache组织举例

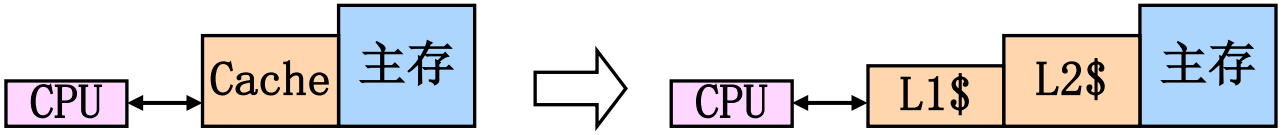
*新的Cache结构：

$T_A = T_{命中} + (1-H)T_{缺失} = T_{命中} + F \times T_{缺失}$

多级Cache—用层次结构代替单级Cache　(L1$是L2$的子集)

├←容量大时，$T_{命中}$较大($\approx nT_C$)、F较低

└←相联度高时，F降低、$T_{命中}$增大

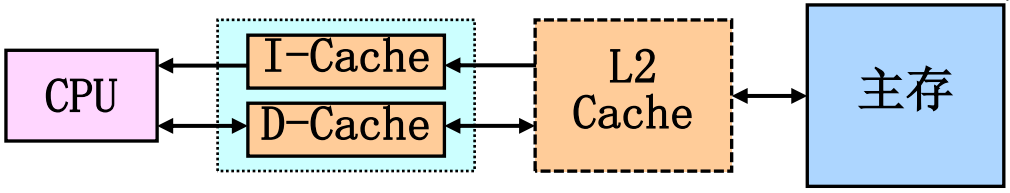


优点：$T_{L1\$命中}$可较小(容量小/结构简单/$\approx T_C$)，$F_{L2\$}$可较低(提高相联度)；

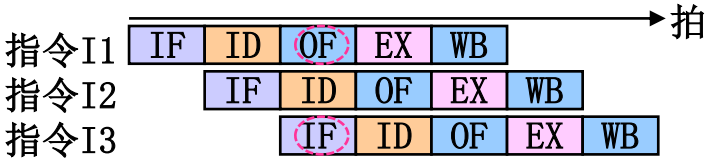
虚-实地址变换与访问L1$可并行(如读出管理信息)　←下节讨论

哈佛结构—用指令Cache＋数据Cache代替联合Cache

└←又称为冯•诺依曼结构



优点：取指令、访问数据可并行(均缺失时才有冲突)　←特别适于流水线



*Pentium的Cache组织：

Cache结构—贯穿式(L1$与L2$级联)2级Cache　←PII为旁侧式

	结构	容量	块大小	映射方式	替换算法	写策略
L1$	哈佛	各8KB	32B	2路组相联	LRU	L1-I$无，L1-D$写回法
L2$	冯•诺依曼	≥256KB				写回法

Cache组成解析—（以L1-D$为例）　←主存按字节编址、地址为32位

内部结构：

组号	标记	状态	缓存块	LRU位	标记	状态	缓存块
0	00000H	11	块数据	1	33333H	00	块数据
1	11111H	00	块数据	0	44444H	10	块数据
…	…	…	…	…	…	…	…
127	22222H	10	块数据	1	55555H	11	块数据
	行0				行1		

地址映射：组数＝(8KB/32B)/2＝128，标记＝32－5－7＝20位

替换算法：每组设置LRU位($\log_2 2$＝1位)　←值互斥、可节省1个

写 策 略：每行设置脏位(状态域低位)　←状态=<有效位,脏位>

思考：上图中，若LRU位=1时替换行1，读44444FE3H号单元的操作过程？

映射：主存地址的标记=44444H、群内块号=1111111B=127(映射到127#组的任一行)

查找：该组各行的V均=1、标记都不是44444H，Cache缺失；

目标块调入该组的行1中(V均=1、LRU=1)，调入前先替换(行1的脏位=1)

*Kunpeng(鲲鹏)单个内核的Cache组织：

内核参数—机器字长为64位，主存按字节编址、容量≤2^{48}B

Cache结构— 贯穿式2级Cache结构，

L1$为哈佛结构，L2$为冯·诺依曼结构

Cache参数—

	容量	块大小	映射方式	替换算法	写策略
L1-I$	64KB	64B	4路组相联	LRU	/
L1-D$	64KB				全写法
L2$	512KB	128B	8路组相联		写回法

Cache组织— --以L1-D$为例

内部结构：有 256 组、每组有 4 行，每行有 V、Tag 等域

地址映射：Tag＝ 48-$\log_2$(64B/1B)-$\log_2$(256)＝34 位

替换算法：每行的LRU位＝ $\log_2$4＝2 位

写 策 略：无需增加硬件

作业3-3：P138—22、24、25、28、29

§3.5 虚拟存储器

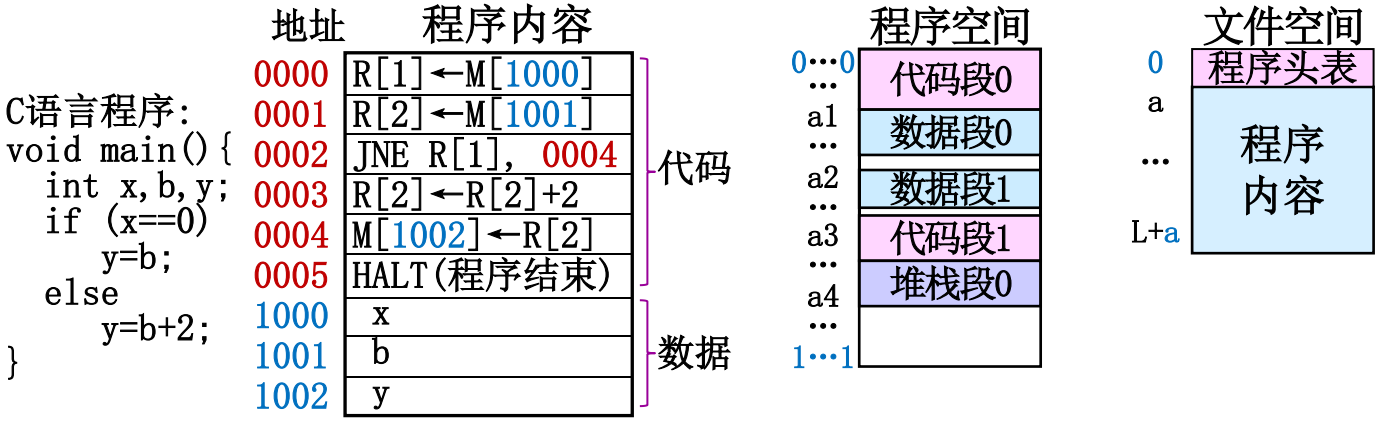
※主要内容：存储管理相关概念，虚存的组成、工作过程、地址变换步骤

*程序的结构&存放：

程序空间—大小＝2^n×主存单元长度，由ISA约定(2^n均＞主存地址空间)

└←装入主存后地址相对不变

程序结构—包含多种段(每种可多个)，各段的访问权限不同(读/写/执行)



程序地址组成：＝<段号，段内地址> 或 段首址+段内地址

└←n-m位 └←m位(始于0) └←n位(如a…b0…0)

程序的存放—文件形式存于辅存中，各段信息放在程序头表中

<类型,长度,程序空间首址,文件空间首址,…>

一、存储管理相关概念 （指主存的管理）

1、主存空间的分配

思考①：程序装入主存后，CPU如何按逻辑地址访问主存？ 先进行地址变换

*分区方式： 主存划分成若干个分区($S_{主存}=\Sigma S_{分区i}$)， ←$S_{分区i}$可不等

每个进程分配到1个分区(程序的空间连续) ←$S_{分区i} \geq S_{程序}$

特　点—

分区类型	固定分区($S_{分区i}$不变)	可变分区(合并+移动分区)
分配方法	选择最接近$S_{进程}$的分区	切割空闲分区
特　点	产生的碎片较大	管理开销较大(合并&移动)

地址变换方法— 物理地址＝基地址＋逻辑地址 ←指分区的基地址

*分页方式： 程序、主存划分成若干个页、页框($S_{页}=S_{页框}$)， ←如$S_{页}$=4KB

└←逻辑地址＝<逻辑页号,页内地址>

每个进程分配到所需全部页框(程序的空间可不连续)

特　点— 碎片小($S_{页} \ll S_{分区}$)， 管理开销小(用页表管理页-页框映射)

地址变换方法— 物理地址＝<物理页号,页内地址>

物理页号＝页表[逻辑页号]

页号	页框号
0	3
1	8
2	1
…	…

思考②：主存分配&管理谁实施？ 引子：程序大小＞主存大小咋办？

2、主存空间的扩充（主存分配的优化）

***覆盖技术：**（适于分区分配方式）

程序划分＝x个不可覆盖程序段＋y个可覆盖程序段

分配空间＝$\Sigma S_{\text{不可覆盖程序段}} + \max(S_{\text{可覆盖程序段}})$ ←不可覆盖常驻

空间扩充：应用程序先将可覆盖程序段调入主存，再执行

└←本来应该OS负责的　　└→覆盖旧的程序段

特点—需应用程序员协助（划分程序段、计算程序段大小）←如程序≤64KB

***交换技术：**（适于分区分配方式）

进程状态＝x个执行态/就绪态、y个等待态（通常时间较长）

空间扩充：等待态可换出主存、就绪态可调入主存 ←部分进程即可

特点—对应用程序员透明（操作系统负责换入/换出）

***虚拟存储技术：**（覆盖&交换技术的发展）

分配空间＝当前活跃的程序段，其余放在辅存 ←基于局部性

空间扩充：访问缺失时，主存与辅存自动交换信息 ←层次结构

特点—对应用程序员透明

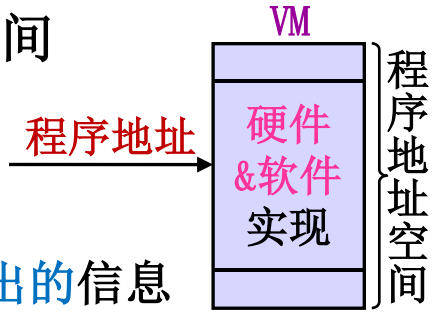
二、虚拟存储器的基本原理

*虚拟存储器(Virtual Memory, VM)：　　—又称虚拟主存

指以透明方式、为程序提供的、远大于主存空间的存储空间

（程序员不管实现）（按程序地址访问）（满足所有应用需求）（速度≈主存）

术语—虚拟地址(逻辑地址)、物理地址，虚拟地址空间

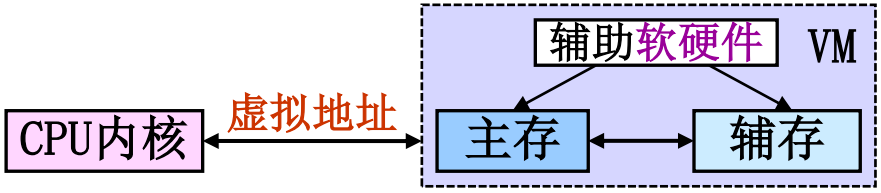


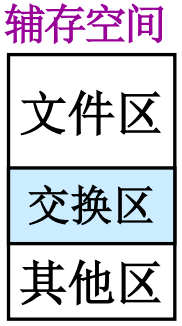
1、虚拟存储器的组成

*基本思想：借助辅存扩充主存的存储空间

└←划出部分空间(交换区)，暂存主存替换出的信息

*基本组成：主存＋辅存构成层次结构，按虚拟地址访问





*实质：是面向软件的存储器模型

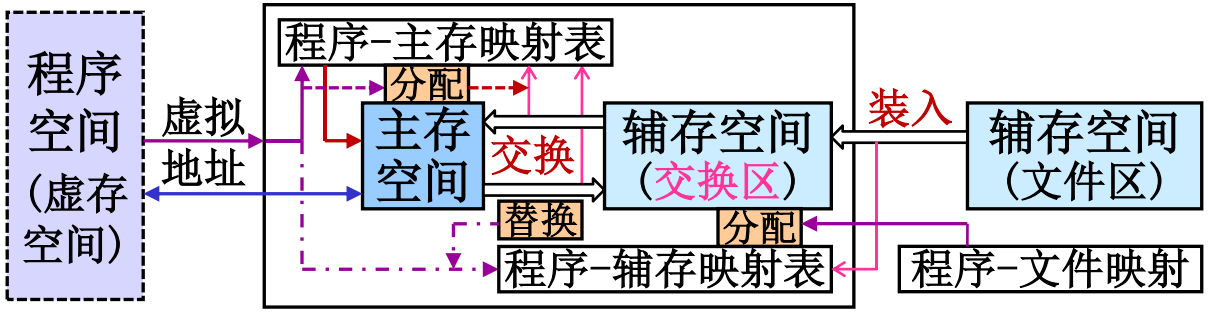
├←程序装入时，活跃部分调入主存、其余放在辅存

└←程序执行时，先地址变换，失败时主存-辅存交换

└←软件编程时，按程序空间进行(不考虑硬件实现)

2、虚拟存储器的工作过程

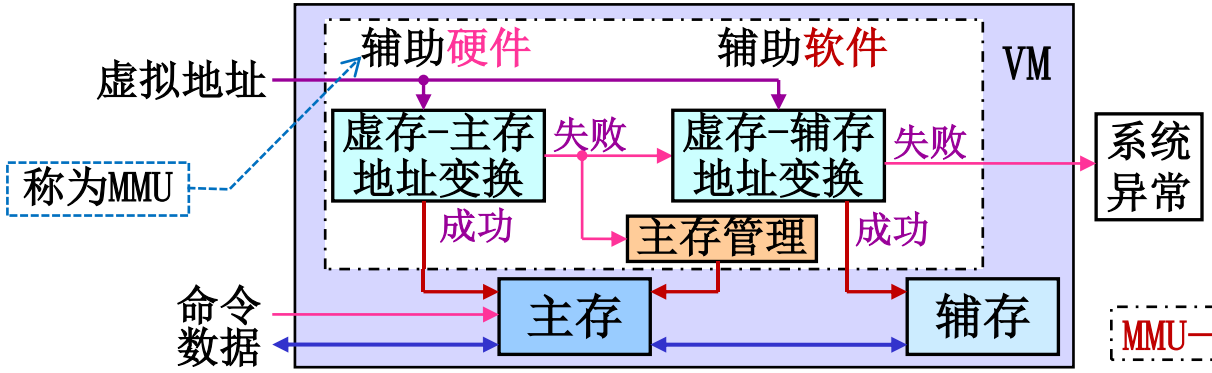
*VM相关地址空间：程序空间，主存空间、辅存空间(交换区&文件区)



交换区：暂存替换出的程序信息
文件区：存放程序文件(与VM无关)

*VM工作过程：地址变换＋[层次管理＋]访问主存

←[]表示可缺省



MMU—Memory Management Unit

思考①：虚存-主存变换为何要用硬件实现？ 需保证访问速度(≈主存)

思考②：虚存-辅存变换为何要用软件实现？ 需访问辅存(速度太慢)、管理表容量太大(成本太高)

思考③：如何从硬件实现转到软件实现？ MMU产生异常请求(见P169图5.1)

三、虚拟存储器的存储管理

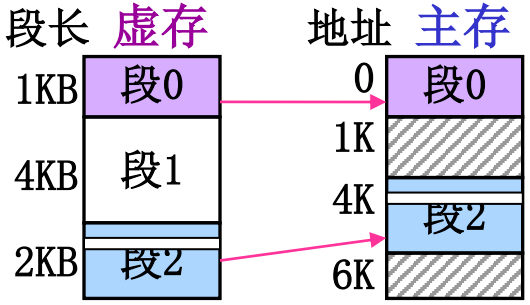
涉及内容—主存-辅存的交换单位，存储空间的管理，地址变换方法

*主存-辅存信息交换单位：程序段(大小可变)、页(大小固定[如4KB])
└←利于进程保护　└←利于主存利用率[碎片小]

*虚存的存储管理方式：段式、页式、段页式(先分段再分页)

1、段式VM　(段式管理)

*管理方式：VM空间按程序结构划分成若干个段，
主存空间按段大小进行分配　←∈分区方式、可多次分配



虚拟地址＝<段号,段内地址>　←类似Cache
└←长度固定(便于管理)

物理地址＝段首址＋段内地址

段号	装入位	段首址	段长
0	1	0K	1K
1	0		4K
2	1	4K	2K

*段表组成：按段号索引，行数＝程序段数

思考：Windows支持64个进程，段号=16位、段表项=8B，段表总空间？ 32MB

段表存放—主存中(基于成本[Cache才4MB])　→段表按段号索引(减少访存)

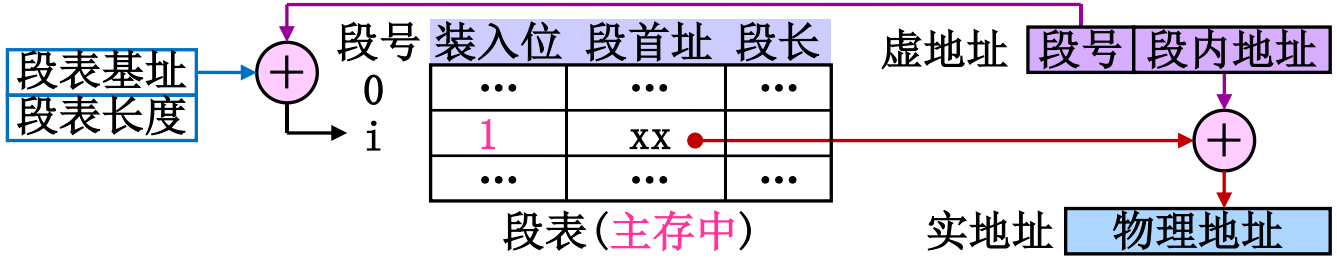
*地址变换过程：

①计算表项地址(=段表基址+段号×表项长度)

②读取表项内容 ←需访问主存1次

③计算物理地址(=段首址+段内地址)、判断是否有异常

└←同分区方式 └←越界/越权等



思考①：实现地址变换的硬件名称？段表基址放在哪里？ MMU，CPU的REG中

思考②：判断异常有什么好处？ 防止进程被破坏，如病毒、BUG

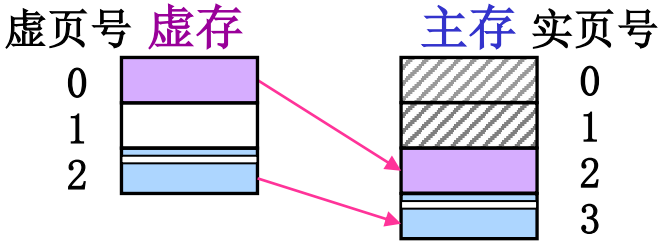
*特点：利于进程共享与保护(面向软件)，主存空间利用率低

└←程序结构基于段 └←碎片较大

2、页式VM （页式管理）

*管理方式：虚存&主存空间按页大小划分成若干个页，

主存空间按页进行分配 ←页可不连续，可多次分配



虚拟地址＝<虚页号, 页内地址>

物理地址＝<实页号, 页内地址>

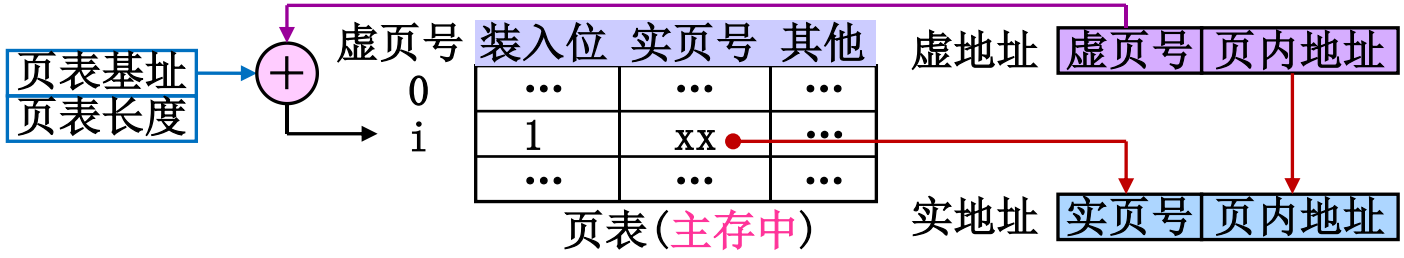
虚页号	装入位	实页号	其他
0	1	2	…
1	0		…
2	1	3	…

*页表组成：按虚页号索引，行数＝程序段数

页表存放—主存中(页表太大/基于成本)

*地址变换过程：计算表项地址，读取表项内容，

拼接成实地址、判断是否有异常(越界)



*特点：主存空间利用率高(面向硬件)，进程共享与保护有限

3、段页式VM （段页式管理）

*管理方式：VM空间先分段、再分页，主存空间仅分页；

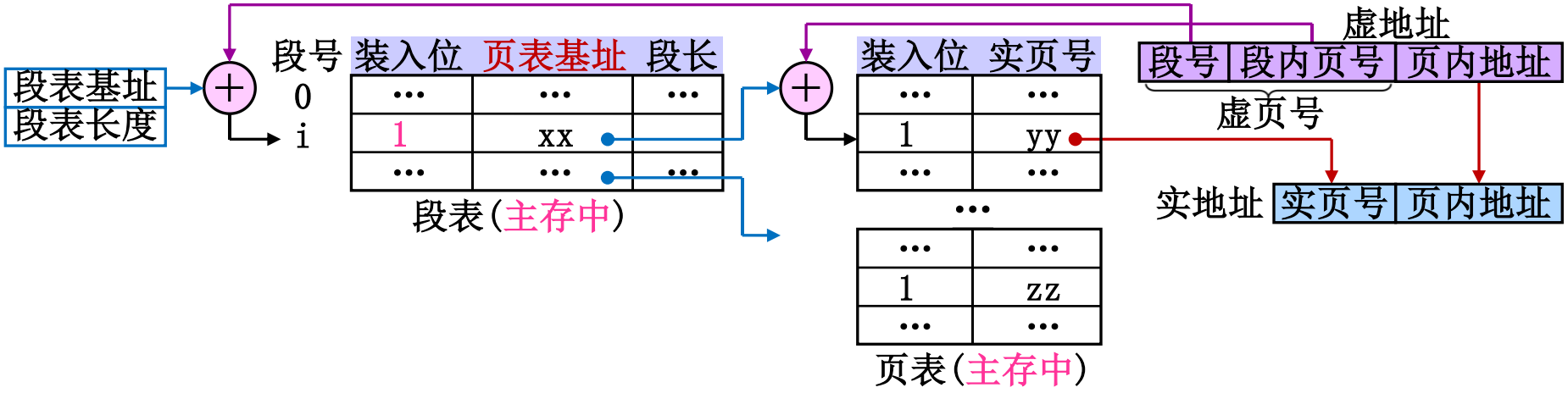
主存空间按页进行分配

虚拟地址＝<段号,段内页号,页内地址>

物理地址＝<实页号,页内地址>

*管理表组成&存放：一个段表＋多个页表，均放在主存中

*地址变换过程：段式、页式地址变换的组合(2次访存)



*特点：兼有段式及页式优点，但需2次访存

四、页式虚拟存储器的实现

相关内容—缓存管理、页表组织、地址变换、缺页处理等
（OS实现）（OS实现）（MMU实现）（OS实现）

1、主存-辅存的缓存组织

（了解软件如何组织）

每个进程有一个虚拟MEM！

背景—进程可用页框数已约定、可调整(缺失率较高时)

*映射方式：全相联映射，映射管理用页表实现(1个/进程)
└←所有页框用位图管理，按序分配

*替换算法：伪LRU算法，状态更新用定时轮询实现
用访问位A表示→┘ ├──────────┘←所有页的A复位
(1/0=近期有/无访问) └→牺牲页：首选A=0的非修改页

*写 策 略：写回法策略 ←写交换区(辅存中)开销太大

*虚存-交换区的映射组织：

常扩展页表实现(增加辅存地址字段)

2、页表(PT)的组织

PT—Page Table　PTE— Page Table Entry

*索引方式：虚页号　←页表放在主存中、减少访存次数

*页表长度：程序所含页数　←CPU需保存表长信息(实现越界检测)

*页表项(PTE)组成：

装入位	实页号	访问位	修改位	禁止缓存位	…

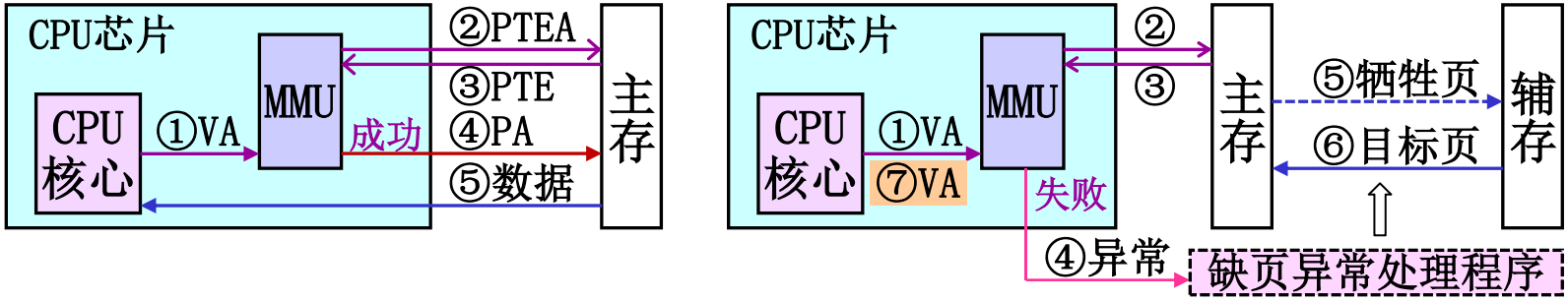
*地址变换方法：实地址＝<页表[虚页号].实页号, 页内地址>

└←＝页表基址＋虚页号×PTE长度

思考：PTE中为何没有标记域？页表＞1页怎么办？　按虚页号索引，多级页表(按实页号才要)(类似段页式)

3、地址变换的实现

*地址变换：MMU实现，计算表项地址＋读取表项＋形成实地址



*缺页处理：OS实现，调入目标页＋返回缺页指令(重新执行)

*地址变换的优化：

性能分析— 需访存1次（$T_{虚存}=2*T_{主存}$）

TLB—Translation Lookaside Table

优化方法— 页表采用层次结构，MMU中设置页表缓冲器（TLB）

（利用访问局部性）　└→地址变换可不访存←┘　└←称为快表

└←同一页的地址变换查同一页表项

TLB的组织—

缓存管理： 组/全相联映射、LRU替换算法、写回策略

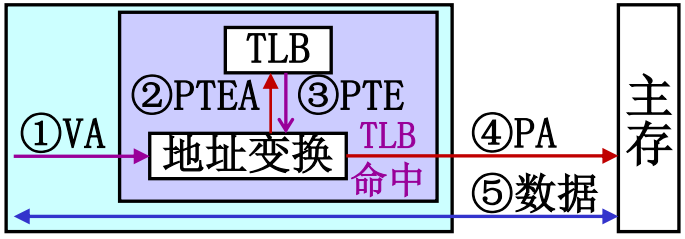
TLB行组织：（类似Cache行，缓存块为页表项）

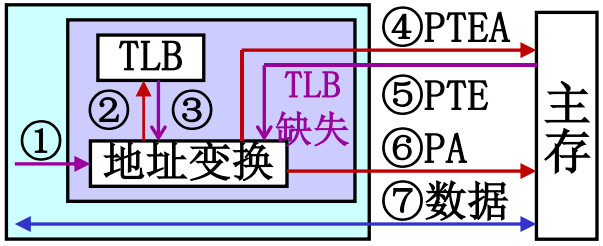
有效位	标记	LRU	修改位	实页号	访问位	…

TLB的大小： 几十行　←$H_{TLB}>>H_{Cache}$

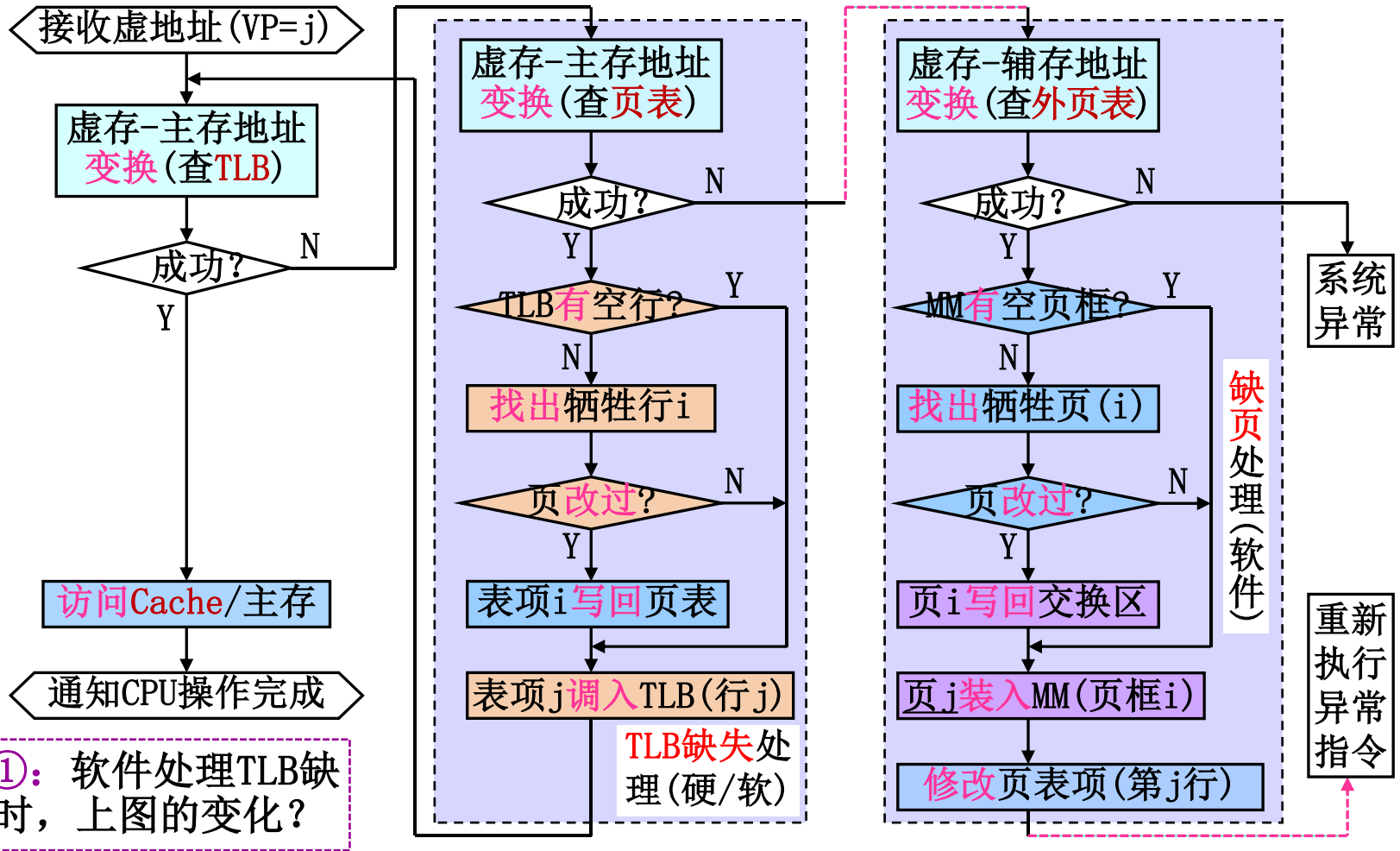
思考：若虚拟地址为48位，按字节编址，页大小为4KB，TLB为64行、4路组相联，其标记、LRU的位数？

地址变换过程的改变— 先访问TLB，TLB缺失时才访问页表





4、页式VM的工作过程



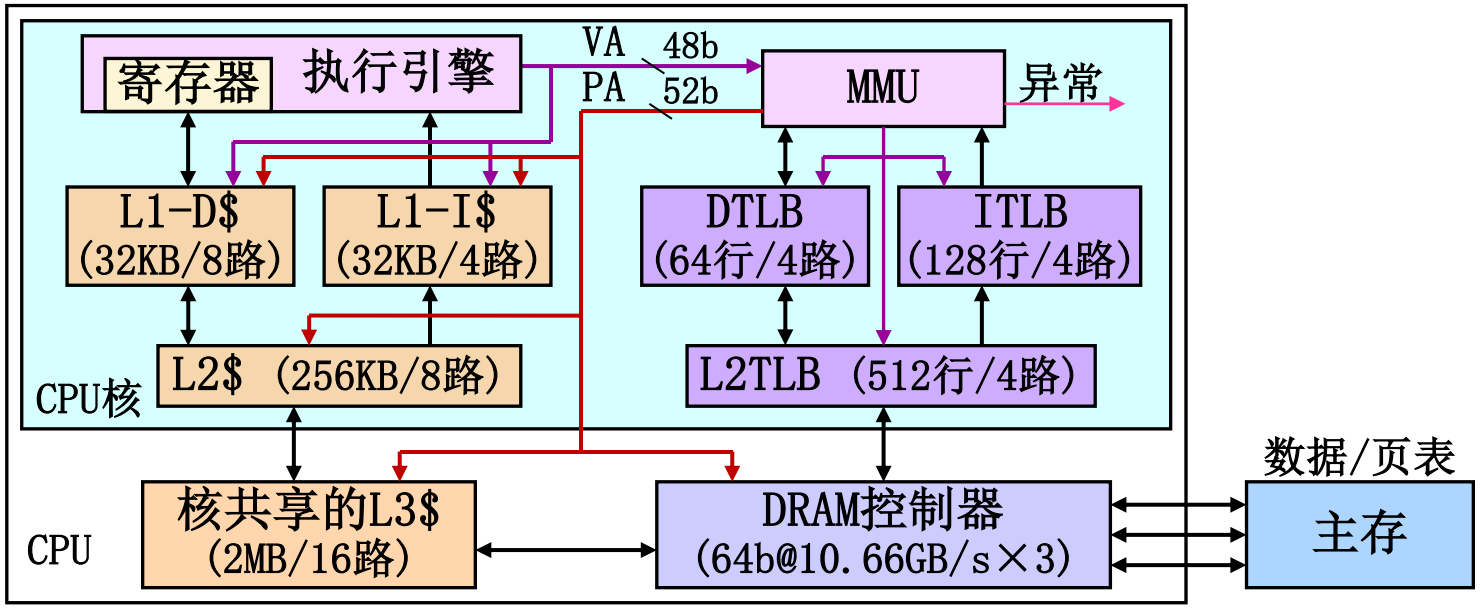
思考①：软件处理TLB缺失时，上图的变化？

思考②：TLB、页表、Cache的命中/缺失组合中，哪些不会发生？

TLB命中、页表缺失，页表缺失、Cache命中

※存储器层次结构综合示例：（以Core i7为例）

层次结构—2级TLB，3级Cache+主存

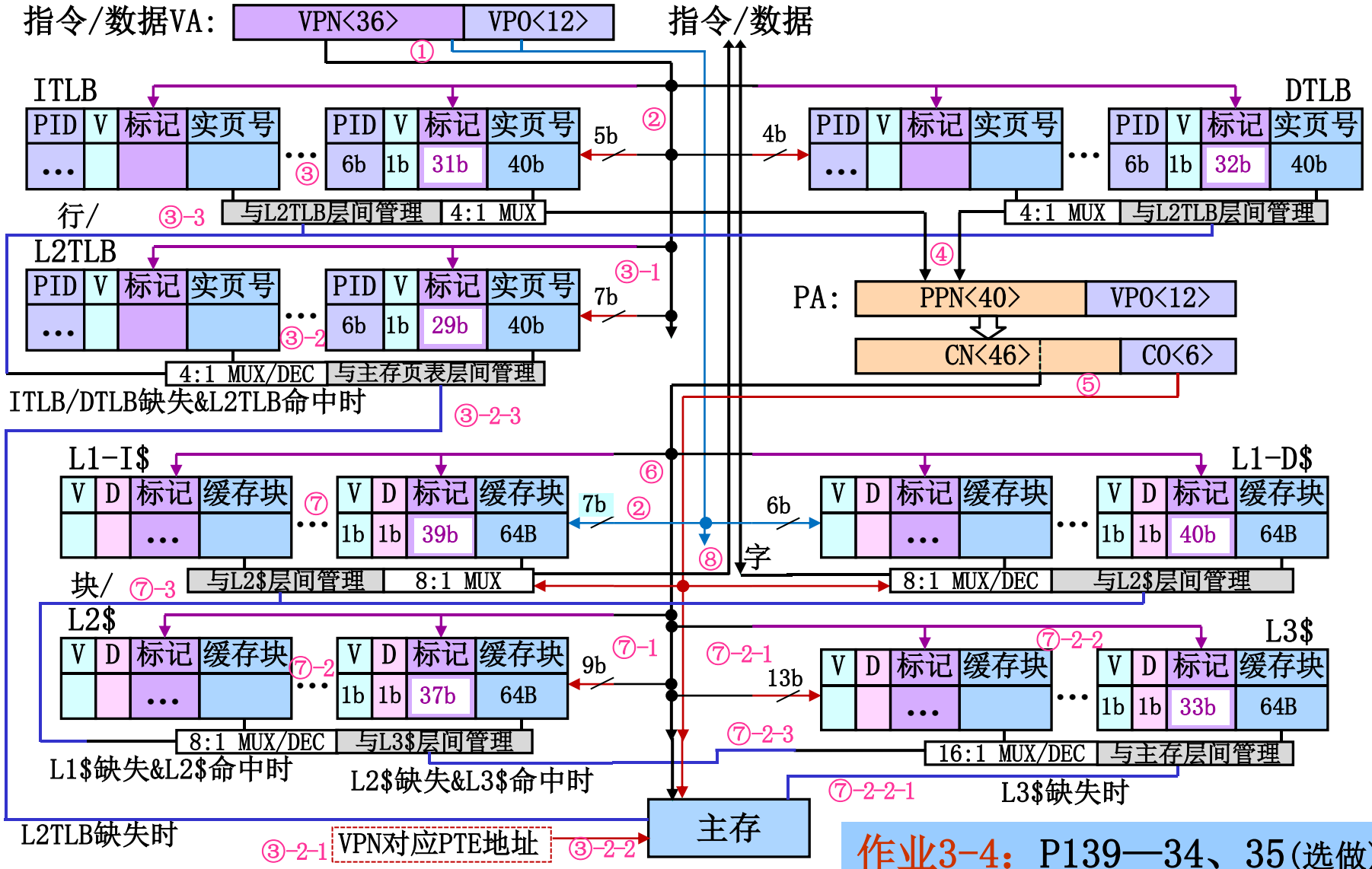


	虚存	TLB	Cache	主存
剩余参数	页式，$S_{页}$＝4KB，$S_{页表项}$＝8B	LRU、写回法，硬件处理缺失	$S_{块}$＝64B，LRU、写回法	DDR3，支持三通道方式

思考①：ITLB、L2$的索引、标记各为几位？ ITLB为5b、31b，L2$为9、37

思考②：MMU的异常信号连接到哪里？ CPU的中断机构

工作过程—地址变换(基于ITLB/DTLB)＋数据访问(基于L1-I$/L1-D$)



作业3-4：P139—34、35(选做)